SHIP: A Shallow and Highly Parallelizable CKKS Bootstrapping Algorithm

Jung Hee Cheon[1,2][0000−0002−7085−2220], Guillaume Hanrot[3][0000−0001−9319−0365]✉, Jongmin Kim[1][0000−0003−2937−3073], and Damien Stehlé[3][0000−0003−3435−2453]

[1] Seoul National University, Seoul, Republic of Korea
{jhcheon, jongmin.kim}@snu.ac.kr
[2] CryptoLab Inc., Seoul, Republic of Korea
[3] CryptoLab Inc., Lyon, France
{guillaume.hanrot, damien.stehle}@cryptolab.co.kr

Abstract. The CKKS fully homomorphic encryption scheme enables efficient homomorphic operations in terms of throughput, but its bootstrapping algorithm incurs a significant latency. In this work, we introduce SHIP, a novel bootstrapping algorithm for CKKS ciphertexts. SHIP enjoys a very shallow homomorphic multiplicative depth compared to state-of-the-art CKKS bootstrapping algorithms. Bootstrapping depth directly impacts the required Ring-LWE modulus, and hence the Ring-LWE degree. The massive depth saving allows us to report the first bootstrapping of CKKS ciphertexts for full-dimensional cleartext vectors in ring degree $N = 2^{13}$, without resorting to an expensive scheme switching to DM/CGGI. SHIP also enjoys great parallelizability, with minimal communication between threads. The combined ring size reduction and high parallelizability lead to very low latency. In ring degree $N = 2^{13}$, our experimental implementation runs in 215ms on a 32-core CPU for real-valued cleartext vectors. This is 2.5x lower than the smallest latency we could observe with the HEaaN library (using 48 cores). For binary cleartext vectors, the latency is lowered to 174ms, which is 2.2x lower than Bae et al [Eurocrypt'24] (with 32 cores).

1 Introduction

The CKKS [CKKS17] fully homomorphic encryption scheme (FHE) provides high-throughput homomorphic computations. Similar to B/FV [Bra12, FV12] and BGV [BGV12], this stems from its ability to perform Single Instruction Multiple Data (SIMD) homomorphic computations on high-dimensional cleartext vectors, using only a few arithmetic operations over a cyclotomic ring typically chosen of the form $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ for N a power of two and q an integer that may vary throughout the computations. Contrarily to B/FV and BGV whose primary cleartexts are vector over finite fields, CKKS natively handles vectors of $N/2$ complex numbers with parameterizable accuracy. The CKKS arithmetic was recently extended to bits and small inte-

gers [DMPS24,BCKS24,CKKL24,BKSS24], notably by relying on complex roots of unity. In this work, we will focus on the CKKS scheme.

The high-throughput efficiency comes at the expense of a relatively complex bootstrapping algorithm [CHK+18a], a procedure that enables further computations on a ciphertext. State-of-the-art CKKS bootstrapping consists of the homomorphic evaluations of the Discrete Fourier Transform (DFT) and its inverse (iDFT), and of a polynomial approximation to the mod-1 function near integers belonging to a large interval. These operations require homomorphic multiplicative depth, and hence a relatively high maximal modulus q, of up to 1700 bits for typical CKKS parameters. The high modulus itself leads to choosing a large ring degree N, to maintain sufficient security. For example, it must be set as $N \geqslant 2^{16}$ for the bit-size just mentioned. Combined with the intrinsic cost of the steps to be performed, this results in a bootstrapping algorithm that remains relatively costly despite numerous improvements (see [CCS19, HHC19, HK20, LLL+21, BMTH21, LLK+22, BTH22, KPK+22, SSKM24], among others). Optimized implementations such as HEaaN [Cry22] perform bootstrapping with $N = 2^{16}$ in a little below 10s in single-thread CPU.

The DM/CGGI [DM15, CGGI16a] FHE schemes achieve a different balance. Their bootstrapping algorithms handle a single cleartext element as opposed to a vector, but do not require homomorphic multiplicative depth, allowing to rely on smaller ring degrees, such as $N = 2^{11}$. This provides lower latency, of the order of 10ms for competitive implementations [CGGI16b, Zam24], but the lack of homomorphic parallelism results in a degraded throughput. Approaches have been put forward to extend DM/CGGI to enable SIMD homomomorphic operations [MS18, LW23a, LW23b, GPL23, MKMS24], but their advantage is currently mostly asymptotic and the achieved practical throughput remains far behind that of CKKS (see [GPL23]). Rather than making DM/CGGI more parallel, we ask the following dual question:

Can we design a CKKS bootstrapping algorithm
that has lower homomorphic multiplicative depth?

State-of-the-art CKKS bootstrapping can be parameterized to require sufficiently low homomorphic multiplicative depth to work with ring degree $N = 2^{15}$ (see, e.g., [CHK+18a]). Enabling bootstrapping in ring degree $N = 2^{14}$ for full-dimensional cleartext vectors was achieved only recently, in the context of bootstrapping bits [BCKS24]. Indeed, such a cleartext type can be handled with smaller-precision computations in bootstrapping, allowing to take a smaller maximal modulus q and hence to lower N. The homomorphic multiplicative depth of CKKS bootstrapping can be decreased by replacing the homomorphic evaluation of the Cooley-Tukey algorithm used for the (i)DFT [CCS19], by a homomorphic matrix-vector multiplication (as was done originally in [CHK+18a]). However, the number of operations in the ring R_q consumed in homomorphic (i)DFT then blows up from $O(\rho \cdot N^{1/\rho})$ where ρ is the number of Cooley-Tukey recursion steps, to $O(N)$. This is prohibitive especially as either the DFT or iDFT has to be performed when ciphertexts have maximal modulus. Another possibility to decrease the homomorphic multiplicative depth of the (i)DFT is to

consider sparsely-packed ciphertexts, i.e., whose underlying cleartexts contain a single or very few scalars as opposed to a large-dimensional vector. As observed in [HHC19], one can then rely on a smaller-dimensional (i)DFT. However, this has a strong negative impact on throughput.

An alternative route was considered in [KDE+24]. In the spirit of the scheme-switching [BGGJ20], the CKKS ciphertext to be bootstrapped is broken into N DM/CGGI ciphertexts, each one of which is bootstrapped using DM/CGGI, before repacking the N output DM/CGGI ciphertexts into a CKKS ciphertext. This last step is achieved by relying on a ring-packing procedure, such as those from [CGGI17,MS18,CDKS21,BCK+23]. This approach requires very low depth: ring degrees $N = 2^{12}$ and $N = 2^{13}$ are respectively put forward in [KDE+24], for high and low bootstrapping failure probabilities. However, the bootstrapping cost balloons to $O(N^2)$ ring operations. Interestingly, it was recently shown in [ACJ24] that the algorithm can prove practical with dedicated hardware, as it enjoys high parallelizability. This highlights the fact that operations count is too coarse as a cost metric: in practice, one rarely runs FHE computations on single-threaded CPUs but rather uses all the CPU cores available, or even goes for other hardware such as GPU and FPGA when possible. We observe that CKKS bootstrapping algorithms based on homomorphic (i)DFT also enjoy some high-grain parallelism, as each recursive step of homomorphic Cooley-Tukey involves rotations of the underlying cleartext vectors that can be parallelized. This parallelizability however degrades when the number ρ of Cooley-Tukey recursion steps increases, which is necessary to lower the overall cost.

1.1 Contributions

We introduce SHIP, a new CKKS bootstrapping algorithm that has low homomorphic multiplicative depth, has an overall cost that is comparable to state-of-the-art CKKS bootstrapping and is highly parallelizable. We implemented SHIP in multi-threaded CPU environments, and our experiments highlight strong performance compared to state-of-the-art CKKS bootstrapping implementations in terms of latency. We report the first bootstrapping of CKKS ciphertexts in ring dimension $N = 2^{13}$ that relies only on CKKS operations (as opposed to [KDE+24] that relies on DM/CGGI bootstrapping). On 32 CPU cores, our experimental code achieves a latency of 215ms, which compares favorably with the smallest bootstrapping latency we could achieve for real-valued cleartexts with the HEaaN library with multi-core CPU, of 624ms using 32 cores. We achieve an even lower 174ms of latency for bootstrapping bits.

The main gain of SHIP over state-of-the-art CKKS bootstrapping is with respect to homomorphic multiplicative depth. CKKS bootstrapping uses an (i)DFT with ciphertexts with highest moduli, and another one at smaller moduli. These are implemented using the Cooley-Tukey algorithm, with ρ_{top} and ρ_{bot} recursion steps, respectively. The cost of homomorphic Cooley-Tukey is $O(\rho \cdot N^{1/\rho})$ arithmetic operations over a ring R_q, which suggests to take large ρ. However, the homomorphic multiplicative depth is linear in ρ and parallelizability decreases

with ρ. As a result, one typically chooses ρ_{top} around 3 or 4, whereas ρ_{bot} can be smaller as the modulus q of the involved ciphertexts is much lower and hence the ring operations are cheaper. Compared to state-of-the-art CKKS, SHIP does not rely on a homomorphic (i)DFT at top-level modulus, and saves ρ_{top} homomorphic multiplicative levels (later referred to as HE-levels). In both approaches, the HE-depth (defined as the number of HE-levels) grows linearly in the Hamming weight h of the secret-key of lowest modulus ciphertexts, but, in practice, this component to HE-depth is lower for SHIP. Overall, SHIP decreases HE-depth from above 12 HE-levels for typical instances of state-of-the-art CKKS bootstrapping, down to 6 HE-levels.

Each HE-level consumes modulus, and the total amount of modulus used by bootstrapping must be lower than the maximal modulus allowed in the given degree N. It turns out that SHIP's HE-levels are shallower, i.e., consume less modulus, than those used in state-of-the-art CKKS bootstrapping. This is because the plaintexts involved in bootstrapping have smaller bit-sizes. Combined together, the smaller HE-depth and shallower HE-levels allow us to push modulus consumption below 200 bits, and to enable bootstrapping (with one level reserved for useful computations) in ring degree $N = 2^{13}$, with 128-bit security.[4]

The modulus consumption reduction of SHIP does not come at the expense of a large cost blow-up, neither in overall cost nor in parallelizability. Let us first focus on the overall cost. Recall that h denotes the Hamming weight of the secret key for low-modulus ciphertexts. In fast bootstrapping algorithms [BTH22], this Hamming weight is often chosen much smaller than N (e.g., $h \in [2^5, 2^7]$ and $N \in [2^{14}, 2^{16}]$). Note that this is also the case for recent BFV/BGV bootstrapping algorithms (see [MHWW24a, MHWW24b, KSS24], even though the idea can be traced back at least to [HS15]). SHIP's overall cost is bounded as $O(h \log(N/h))$ ring operations at top modulus, and $O(\rho_{\text{bot}} \cdot N^{1/\rho_{\text{bot}}})$ ring operations at low modulus (in practice, we will set $\rho_{\text{bot}} = 1$). Oppositely, state-of-the-art CKKS bootstrapping requires $O(\rho_{\text{top}} \cdot N^{1/\rho_{\text{top}}} + \log h)$ ring operations at high modulus and $O(\rho_{\text{bot}} \cdot N^{1/\rho_{\text{bot}}})$ ring operations at low modulus.[5] Setting ρ_{top} to a large value allows CKKS bootstrapping to outperform SHIP, but, in practice, this is in part compensated by the fact that SHIP's ring size $N \log q$ is much smaller (thanks to the limited modulus consumption).

For the sake of simplicity, we measure parallelizability by the number of sequential ring operations involved by the algorithm, assuming an unlimited number of processors. This models latency, in a scenario with many cores and free communications. Note that ring operations can themselves be highly parallelized using double-CRT techniques (see [BEHZ16, CHK+18b]), but, as this is the case

[4] We exclusively focus on 128-bit security, as measured using the lattice estimator [APS15].

[5] The term $\log h$ comes from need of an approximation of the mod-1 function near integers for an interval of width $O(h)$ – or $O(\sqrt{h})$ if heuristic correctness suffices. This can be achieved by approximating $\cos(2\pi(x - 1/2^{k-2}))$ over $[-1, 1]$ and using $k = O(\log h)$ times the *double angle formula* $\cos(2u) = 2\cos^2(u) - 1$ to obtain an approximation of $\sin(2\pi 2^k x)$ over $[-1, 1]$.

for all considered schemes, we omit this aspect and stay at the ring level. The parallel depth of SHIP is bounded as $O(\log N + \rho_{\text{bot}})$ ring operations, whereas the parallel depth of classical CKKS bootstrapping is $O(\rho_{\text{top}} + \rho_{\text{bot}} + \log h)$. In practice, the situation is favorable to SHIP as the dominant part of the cost can be parallelized at a very coarse grain into h independent computations, whereas the top-moduli Cooley-Tukey either requires far more processors or more communication rounds.

Table 1. Summary of the experimental results of performing bootstrapping for real numbers using SHIP or the baseline CKKS bootstrapping method in the HEaaN library [Cry22]. The SHIP results are given for a latency-optimized parameter set ('latency-opt') and for a throughput-optimized parameter set ('throughput-opt'). More details and further experimental results are given in Section 5.4.

Scheme & parameter choice	N	HE-level (mult)	HE-level (boot)	Precision (bits)	Latency 1-core	Latency 32-core (speedup)
SHIP (latency-opt)	2^{13}	1	6	4.45	**3.02s**	**215ms** (14.1×)
SHIP (throughput-opt)	2^{15}	9	6	18.1	29.6s	1.63s (18.1×)
CKKS-HEaaN	2^{15}	1	14	16.7	3.44s	624ms (5.52×)
CKKS-HEaaN	2^{16}	9	15	22.0	8.68s	1.09s (7.97×)

The experimental results, which are highlighted in Table 1, show that, thanks to the reduced HE-level consumption of bootstrapping, SHIP enables constructing a lightweight parameter set with $N = 2^{13}$ that enjoys much lower latency. It reaches 215ms when using 32 CPU cores. Especially, due to the high parallelizability of our bootstrapping algorithm, SHIP shows decent multi-core scalability, with 14.1–18.1× improvements when increasing the number of cores from 1 to 32. Meanwhile, the original CKKS bootstrapping method merely shows 5.52–7.97× improvements. If we alternatively optimize the parameter choices for throughput (defined as in Equation (9)), SHIP shows comparable performance to the original CKKS bootstrapping method. For the same ring degree $N = 2^{15}$, the latency is slightly lower, but much more multiplicative levels are attainable with SHIP to result in higher throughput. It also shows comparable throughput to HEaaN when the latter uses $N = 2^{16}$.

1.2 Technical Overview

The goal of CKKS bootstrapping is to regain modulus, i.e., to take a bottom-level ciphertext and map it to a ciphertext with higher modulus with the same underlying cleartext (up to some accuracy, in the case of real or complex numbers). Let q_0 be a small modulus. Assume that we are given a ciphertext $\mathsf{ct} = (a, b) \in R_{q_0}^2$ that decrypts under a secret key $s \in R$ to a polynomial $m \in R$, which itself encodes a cleartext vector $(\mu_i)_{0 \leqslant i < N/2} \in [-1, 1]^{N/2}$ in its coefficients:

$$as + b = m \bmod q_0 \quad \text{and} \quad m(X) \approx \sum_{0 \leqslant i < N/2} \frac{q_0 \mu_i}{\gamma} X^i \ , \tag{1}$$

where $\gamma \gg 1$ is an encoding gap. In the CKKS terminology, we will consider a real bootstrapping rather than a complex bootstrapping (note that complex bootstrapping can be obtained by real bootstrapping, using homomorphic complex conjugation). The gap $\gamma \gg 1$ ensures that $\|m\| \ll q_0$.

Let $\omega = \exp(2\iota\pi/q_0) \in \mathbb{C}$. Our goal is to compute a ciphertext $\mathsf{ct}' = (a', b') \in R^2_{Q'}$ with $Q' \gg q_0$ under a possibly different secret key s', such that ct' decrypts to a plaintext m' that encodes $(\frac{\gamma}{2\pi}\mathrm{Im}(\omega^{m_i}))_{0 \leqslant i < N/2}$ in its slots:

$$a's' + b' = m' \bmod Q' \quad \text{and} \quad m' \approx \Delta \cdot \mathrm{iDFT}(\omega^{m_0}, \ldots, \omega^{m_{N/2-1}}) \ , \tag{2}$$

where Δ is the scaling factor whose purpose is to drive the cleartext precision. We note that going from Equation (1) to Equation (2) exactly corresponds to the original description of CKKS bootstrapping in [CHK+18a]. SHIP differs in how it obtains ct' from ct. Before explaining how it computes ct', we briefly recall why obtaining ct' allows us to achieve bootstrapping. For $0 \leqslant i < N/2$, we have:

$$\left|\mathrm{Im}(\omega^{m_i}) - \frac{2\pi}{q_0} m_i\right| = \left|\sin\left(\frac{2\pi m_i}{q_0}\right) - \frac{2\pi}{q_0} m_i\right| \leqslant \frac{(2\pi)^3}{6q_0^3} m_i^3 \leqslant \frac{(2\pi)^3}{6\gamma^3} \ .$$

Using the relation $m_i \approx q_0 \mu_i / \gamma$, we obtain

$$\left|\frac{\gamma}{2\pi}\mathrm{Im}(\omega^{m_i}) - \mu_i\right| \lesssim \frac{(2\pi)^2}{6\gamma^2} \ .$$

By using homomorphic conjugation and appropriate scaling, a ciphertext as in Equation (2) gives a slot-encoded ciphertext of high modulus for the cleartext vector $(\mu_i)_{0 \leqslant i < N/2}$, with a precision that can go up to $\approx 2 \log \gamma$. After such a ciphertext has been computed, we can perform native CKKS homomorphic computations on it. Finally, to get back to the format of Equation (1) and prepare for a subsequent bootstrapping, we can perform a homomorphic (i)DFT as in conventional CKKS bootstrapping. This can be achieved with Cooley-Tukey, but that involves HE-depth and modifies the order of the slots (in a bit-reversed way), or with a matrix-vector multiplication.

Bootstrapping with blind rotations. We now explain how SHIP obtains ct' as in Equation (2). From Equation (1) and using the fact that $\omega^{q_0} = 1$, we obtain, for all $0 \leqslant i < N/2$:

$$\omega^{m_i} = \omega^{(as+b)_i} = \omega^{b_i + \sum_{0 \leqslant j < N} a_{i-j} s_j \alpha_{ij}} = \omega^{b_i} \cdot \prod_{0 \leqslant j < N} \omega^{a_{i-j} s_j \alpha_{ij}} \ ,$$

where the indices of a are taken modulo N and $\alpha_{ij} = 1$ if $0 \leqslant j \leqslant i$ and $\alpha_{ij} = -1$ otherwise. Note that the α_{ij}'s stem from the reduction modulo $X^N + 1$. As CKKS enables homomorphic operations on vectors rather than scalars, we interpret this identity simultaneously for all values of i:

$$(\omega^{m_i})_{0 \leqslant i < N/2} = \left(\omega^{b_i}\right)_{0 \leqslant i < N/2} \odot \bigodot_{0 \leqslant j < N} (\omega^{a_{i-j} s_j \alpha_{ij}})_{0 \leqslant i < N/2} \ , \tag{3}$$

where the notation $\odot$ refers to the entry-wise product. We observe that, since the non-zero s_j's are in $\{-1, 1\}$, the vector $(\omega^{a_{i-j} s_j \alpha_{ij}})_{0 \leqslant i < N/2}$ is, up to conjugation of certain slots, part of a rotation by j positions of the vector $(\omega^{a_i})_{0 \leqslant i < N}$.

To evaluate Equation (3), we rely on a procedure that we call blind rotation, by analogy with the key component of DM/CGGI bootstrapping [DM15, CGGI16a], and we let this primitive be denoted by $\mathsf{BlindRot}$. It homomorphically rotates the coordinates of a cleartext by an amount that is provided in encrypted form. More precisely, it should satisfy the following specifications:

> $\mathsf{BlindRot}$: it takes as inputs a ciphertext with underlying cleartext $\mathrm{pt} = (\mathrm{pt}_0, \mathrm{pt}_1, \ldots, \mathrm{pt}_{N/2-1}) \in \mathbb{C}^{N/2}$ (in slots) and an encrypted index j; it outputs a ciphertext whose underlying cleartext is $\mathsf{Rot}_j(\mathrm{pt}) = (\mathrm{pt}_{N/2-j}, \mathrm{pt}_{N/2-j+1}, \ldots, \mathrm{pt}_{N/2-j-1}) \in \mathbb{C}^{N/2}$ (in slots).

For the moment, we assume we have such a procedure, and will explain how to instantiate it later. To evaluate Equation (3), we will use a blind rotation for every index j such that $s_j \neq 0$ (i.e., such that $s_j = \pm 1$). To minimize the number of calls to $\mathsf{BlindRot}$, it is preferable to have a secret key s that has a small Hamming weight h. Overall, we aim at evaluating Equation (3) by:

1. computing $(\omega^{a_{i-j} s_j \alpha_{ij}})_{0 \leqslant i < N/2}$ for all j such that $s_j \neq 0$ — this is implemented with h parallel calls to $\mathsf{BlindRot}$;
2. evaluating the product with a binary tree — this costs $h + 1$ homomorphic multiplications and has HE-depth $\lceil \log(h + 1) \rceil$.

Masking. We observe that several difficulties arise when implementing the first one of these two steps, even assuming the existence of a $\mathsf{BlindRot}$ algorithm. First, there are N coefficients a_i, but only $N/2$ slots per ciphertext. Second, Rot_j does not handle the sign of a non-zero s_j. Finally, Rot_j rotates coordinates without adding a sign when taken the last coordinate in first position, as opposed to multiplication modulo $X^N + 1$ which leads to the α_{ij}'s of Equation (3). These difficulties are handled by storing the a_i's in two ciphertexts (one for $i < N/2$ and one for $i \geqslant N/2$), and by using masks. We prove that for every j, the vector $(\omega^{a_{i-j} s_j \alpha_{ij}})_{0 \leqslant i < N/2}$ can be obtained by:

- multiplying the first and second halves of $(\omega^{a_i})_i$ and $(\omega^{-a_i})_i$ by appropriate masks that depend on s_j and j;
- summing the resulting four ciphertexts;
- rotating the result by j positions, using $\mathsf{BlindRot}$.

Observe that in the first of these three steps, the four halves of $(\omega^{a_i})_i$ and $(\omega^{-a_i})_i$ are in clear (the a_i's are part of the ciphertext we want to bootstrap, but their meaning here is irrelevant). Oppositely, the masks are encrypted as they contain information on the secret key: they are part of the bootstrapping key. This plaintext-ciphertext multiplication can be performed without consuming any HE-depth, using the so-called auxiliary prime (normally reserved for key switching) as scaling factor. The sum does not require any HE-depth either, and we will see that this is also the case for the blind rotation.

Rotating blindly. We now explain how to implement rotation by an encrypted amount, with inputs/outputs as specified above. Assume we are given a ciphertext ct with underlying cleartext $\mathrm{pt} = (\mathrm{pt}_0, \mathrm{pt}_1, \ldots, \mathrm{pt}_{N/2-1}) \in \mathbb{C}^{N/2}$ (in slots). We want to rotate the underlying cleartext vector by j positions, where j is encrypted. In CKKS, slot rotation is obtained by evaluating the ciphertext in X^{5^j}, and then switching the secret key from $s(X^{5^j})$ to $s(X)$. (Note that 5 is a generator of the group $\mathbb{Z}_N^\times/\pm 1$.) Here, we cannot rely on this approach as j is encrypted. Instead, we write j as a binary combination of fixed values $j = \sum_{0 \leqslant k < \log N} j_k 2^k$ with $j_k \in \{0, 1\}$ that is provided in encrypted form for all k. We rotate by j indices by sequentially rotating by 2^k or keeping the current ciphertext for $k = 0, 1, \ldots, \log N$, where the choice of rotating or keeping is based on the value of the encrypted j_k.

Rotating by 2^k or keeping the current ciphertext is implemented using a basic step which we call half-mux-rotate. When called with an encrypted input bit set to 1, then the output ciphertext has an underlying cleartext that has been rotated by 2^k positions. If otherwise the encrypted input bit is 0, then the output ciphertext decrypts to 0. It can be viewed as a tweak of CKKS cleartext rotation, where the encrypted bit is multiplied by the rotated secret key in the key-switching key. Using two calls to half-mux-rotate with rotation amounts and encrypted bits respectively set 2^k with j_k and 0 with $1 - j_k$, we obtain a mux-rotate: if the encrypted bit j_k is 1, then the output ciphertext has an underlying cleartext that has been rotated by 2^k positions; if the encrypted bit j_k is 0, then the output ciphertext has an underlying cleartext that is the same as that of the input.

Cost analysis. The largest component to the cost of SHIP is coming from the h blind rotations by the non-zero indices of the secret key. Each one of them requires $O(\log N)$ ring operations, an estimate which can be decreased to $O(\log(N/h))$ under an equidistribution assumption for the non-zero coefficients of the secret key discussed in Section 5.2. These blind rotations can be performed in parallel, resulting in a parallel depth of $O(\log N)$ or $O(\log(N/h))$ ring operations. Finally, they do not require any HE-level (and neither does the masking step).

After the blind rotations, the product tree consumes $O(\log h)$ HE-levels. Its overall cost is $O(h)$, and its parallel depth is $O(\log h)$.

Implementation. We have implemented SHIP using HEaaN [Cry22], targeting a multi-threaded environment. For practical efficiency, we rely on a number of optimizations.

- We introduce a level-saving trick that uses the CKKS auxiliary modulus, designed for key-switching, to perform a plaintext-ciphertext product.
- We propose a simpler blind rotation method based on the column method for matrix-vector multiplication. It is asymptotically less efficient, but involves cheaper plaintext-ciphertext multiplications instead of rotations. Hybridizing it with the mux-method described above proves effective in practice.

- We describe a variant of half-mux-rotate using base-B decomposition rather than binary decomposition (for $B \geqslant 2$); when combined with hoisting [HS18, BMTH21] it also brings a significant practical improvement.

Other related works. Relying on roots of unity to natively enable modular reduction in bootstrapping has been considered in various works. This is notably the case of the DM/CGGI [DM15,CGGI16a] schemes, whose primary differences with our approach are that only a single cleartext element is handled by a ciphertext and modular reduction is by the ring degree N. In [OPP23], using a root of unity in a finite field was considered to compute the product with the secret key in its exponent as in DM/CGGI, but this scheme has high homomorphic multiplicative depth and also handles a single cleartext element. As in our work, classical CKKS bootstrapping [CHK+18a] also relies on roots of unity encoded in slots to perform modular reduction, but it is used to remove the modulus of the underlying cleartext (by computing a complex exponential) rather than to compute the product with the secret key in the exponent.

Notations. We use a bold font for vectors. For $\mathbf{v}, \mathbf{v}' \in \mathbb{C}^{N/2}$, we let $\odot$ denote the ring multiplication of $\mathbb{C}^{N/2}$, namely $\mathbf{v} \odot \mathbf{v}' = (v_i \cdot v'_i)_{0 \leqslant i < N/2}$. For $\mathbf{v} \in \mathbb{C}^{N/2}$ and $0 \leqslant j < N/2$, we let $\mathsf{Rot}_j(\mathbf{v})$ denote the vector $(v_{i-j \bmod N/2})_{0 \leqslant i < N/2}$. We define $R = \mathbb{Z}[X]/(X^N + 1)$ for any $N \geqslant 2$ that is a power of 2, and $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ for any integer $q \geqslant 2$. All our logarithms are in base 2. We let ι denote a complex root of -1.

2 Reminders

In this section, we provide necessary reminders on CKKS.

2.1 Native CKKS Cleartexts and Operations

CKKS is an FHE scheme that operates on vectors of complex numbers.

Encoding and decoding. The ability of CKKS to perform coordinate-wise operations is based on the use of DFT in its encoding mechanism. Encoding maps cleartexts (the data of interest) to plaintexts (inputs to encryption). For $0 \leqslant j < N/2$, let $\zeta_j = \exp(\iota 5^j \pi/N)$ be a system of pairwise non-conjugate primitive $(2N)$-th roots of unity. DFT is the map $m \mapsto (m(\zeta_j))_{0 \leqslant j < N/2}$, and iDFT is the inverse map. Given an integer Δ called the scaling factor and a modulus Q, on input $\boldsymbol{\mu} \in \mathbb{C}^{N/2}$, CKKS slot encoding builds a plaintext, whereas the encoding map comes with a corresponding decoding map such that $\mathsf{Dcd}(\mathsf{Ecd}(\boldsymbol{\mu})) \approx \boldsymbol{\mu}$:

$$\begin{aligned} \mathsf{Ecd} : \mathbb{C}^{N/2} &\to R_Q & \qquad \mathsf{Dcd} : R_Q &\to \mathbb{C}^{N/2} \\ \boldsymbol{\mu} &\mapsto \lfloor \Delta \cdot \mathrm{iDFT}(\boldsymbol{\mu}) \rceil & m &\mapsto \Delta^{-1} \cdot \mathrm{DFT}(m) \end{aligned}$$

Note that parameters are always chosen so that plaintexts have coordinates whose magnitudes are small compared to Q, so we could replace R_Q by R in the above. However, encoding and decoding typically prepare and precede encryption and decryption, respectively, and the latter involve a modular ring.

For this reason, in practice, encoding and decoding often contain some RNS arithmetic [BEHZ16] parameterized by Q.

Basic functionalities. CKKS uses Ring-LWE ciphertexts [SSTX09,LPR10]. As such, it has key generation, encryption and decryption algorithms that are typical of Ring-LWE encryption schemes. We refer to the literature for discussions on the distributions of the diverse ring elements.

- KeyGen: on input 1^λ, where λ is the security parameter, returns a (ternary) secret key $\mathsf{sk} \in R$ and a public key $\mathsf{pk} = (a, a \cdot \mathsf{sk} + e) \in R_Q^2$;
- Dec: on input a ciphertext $\mathsf{ct} = (a, b) \in R_Q$ and a secret key $\mathsf{sk} \in R$, returns $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) := a \cdot \mathsf{sk} + b \bmod Q \in R$;
- Enc: on input a plaintext $\mathrm{pt} \in R_Q$, returns a ciphertext $\mathsf{ct} \in R_Q^2$ such that $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) \approx \mathrm{pt}$, where sk refers to the secret key corresponding to pk.

We will also use key switching.

- SwkGen: given two keys $\mathsf{sk}, \mathsf{sk}'$ and two integers P, Q, returns a switching key $\mathsf{swk}_{\mathsf{sk}' \to \mathsf{sk}} \in R_{PQ}^2$ which is an encryption of $P \cdot \mathsf{sk}'$ under the key sk;
- $\mathsf{KeySwitch}$: given a switching key $\mathsf{swk}_{\mathsf{sk}' \to \mathsf{sk}} \in R_{PQ}^2$ and a ciphertext $\mathsf{ct} \in R_Q^2$ encrypted under sk', returns $\mathsf{ct}' = \mathsf{KeySwitch}_{\mathsf{swk}_{\mathsf{sk}' \to \mathsf{sk}}}(\mathsf{ct}) \in R_Q^2$ such that $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) \approx \mathsf{Dec}_{\mathsf{sk}'}(\mathsf{ct}')$.

Modulus, rescaling and levels. For a plaintext $\mathrm{pt} \in R_Q$ or ciphertext $\mathsf{ct} \in R_Q^2$, the integer Q is called the modulus of the plaintext or ciphertext, respectively. The modulus is modified when the ciphertext undergoes a rescale operation:

- $\mathsf{Rescale}$: on input $\mathsf{ct} = (a, b) \in R_Q^2$ and $\Delta | Q$, returns $(\lfloor a/\Delta \rceil, \lfloor b/\Delta \rceil) \in R_{Q/\Delta}^2$.

In view of the identity

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{Rescale}_\Delta(\mathsf{ct})) \approx \Delta^{-1} \cdot \mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) \ ,$$

we see that the $\mathsf{Rescale}$ operation operates on the underlying plaintext by dividing it by Δ. The main role of $\mathsf{Rescale}$ stems from the encoding operation: as $\mathsf{Ecd}(\mu_1) \approx \Delta \cdot \mathrm{iDFT}(\mu_1)$ and $\mathsf{Ecd}(\mu_2) \approx \Delta \cdot \mathrm{iDFT}(\mu_2)$, we have

$$\mathsf{Ecd}(\mu_1 \odot \mu_2) \approx \Delta^{-1} \cdot \mathsf{Ecd}(\mu_1) \cdot \mathsf{Ecd}(\mu_2) \ .$$

Multiplying messages thus requires rescaling the result of the multiplication of the encoded cleartexts, which, in the ciphertext domain is in turn achieved by rescaling the corresponding ciphertext.

Modulus can be reduced by sending $\mathsf{ct} \in R_Q^2$ onto $\mathsf{ct} \bmod Q' \in R_{Q'}^2$ for $Q' | Q$. On the contrary, when the modulus of a given ciphertext reaches a certain minimal value Q_{S2C} it must undergo a specific procedure, named bootstrapping, to restore modulus to its maximal value Q_{top} in order to allow for further multiplications. Bootstrapping itself uses a significant amount of modulus (see Section 2.2), from Q_{S2C} down to q_0 and from Q_{top} to Q_{comp}. The part of the budget comprised in $[Q_{\mathsf{S2C}}, Q_{\mathrm{comp}}]$ is available for computations.

In practice, most implementations of CKKS use the so-called RNS-CKKS variant [CHK+18b] based on the RNS representation [BEHZ16]; the latter relies on moduli of the form $Q_r = \prod_{0 \leqslant i \leqslant r} q_i$ for some $0 \leqslant r \leqslant \ell$, with $Q_{\text{top}} = Q_\ell$. In that case, a ciphertext at modulus Q_r is said to be at level r. The q_i's do not need to be of the same size, hence level consumption is a slightly coarser, but simpler, measure of the modulus consumption of a homomorphic algorithm.

Homomorphic operations. The CKKS system provides a number of homomorphic operations on encrypted data. We shall make use of the following.

Addition. On input $\mathsf{ct}_0 = (a_0, b_0)$ and $\mathsf{ct}_1 = (a_1, b_1) \in R_Q^2$ encrypted under the same secret key sk, $\mathsf{Add}(\mathsf{ct}_0, \mathsf{ct}_1) \in R_Q^2$ is a ciphertext such that

$$\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{Add}(\mathsf{ct}_0, \mathsf{ct}_1))) \approx \mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_0)) + \mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_1)) \ .$$

Plaintext-ciphertext multiplication. On input $\mathrm{pt} \in R_Q$ and $\mathsf{ct} = (a, b) \in R_Q^2$, PCMult returns a ciphertext over $R_{Q/\Delta}$ such that

$$\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{PCMult}(\mathrm{pt}, \mathsf{ct})) \approx \mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathrm{pt})) \odot \mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct})) \ .$$

Ciphertext-ciphertext multiplication. On input $\mathsf{ct}_0, \mathsf{ct}_1 \in R_Q^2$ with scaling factor Δ and given a relinearization key $\mathsf{relk} = \mathsf{swk}_{\mathsf{sk}^2 \to \mathsf{sk}}$, CCMult returns a ciphertext $\mathsf{CCMult}_{\mathsf{relk}}(\mathsf{ct}_0, \mathsf{ct}_1) \in R_{Q/\Delta}^2$ such that

$$\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{CCMult}_{\mathsf{relk}}(\mathsf{ct}_0, \mathsf{ct}_1)) \approx \mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_0)) \odot \mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_1)) \ .$$

Rotations. On input $\mathsf{ct} = (a, b) \in R_Q^2$ encrypted under a secret key $\mathsf{sk} \in R$, an index $0 \leqslant r < N/2$ and a rotation key $\mathsf{rotk}_r = \mathsf{swk}_{\mathsf{sk}(X^{5^r}) \to \mathsf{sk}}$, HRot returns a ciphertext $\mathsf{ct}' = \mathsf{KeySwitch}_{\mathsf{rotk}_r}(a(X^{5^r}), b(X^{5^r}))$ such that

$$\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}')) \approx \mathsf{Rot}_r(\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}))) \ .$$

Conjugation. On input $\mathsf{ct} = (a, b) \in R_Q^2$ encrypted under a secret key $\mathsf{sk} \in R$ and a conjugation key $\mathsf{conjk} = \mathsf{swk}_{\mathsf{sk}(X^{-1}) \to \mathsf{sk}}$, HConj returns a ciphertext $\mathsf{ct}' = \mathsf{KeySwitch}_{\mathsf{conjk}}(a(X^{-1}), b(X^{-1}))$ such that

$$\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}')) \approx \mathsf{Conj}(\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}))) \ .$$

2.2 State-of-the-Art CKKS Bootstrapping

When a ciphertext $\mathsf{ct} = (a, b)$ reaches a certain minimal modulus, a larger modulus must be restored in order to perform further operations. This restoration is the purpose of bootstrapping. It takes as input $\mathsf{ct} \in R_{Q_{\mathsf{S2C}}}^2$ and returns $\mathsf{ct}' \in R_{Q_{\text{comp}}}^2$ such that $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) \approx \mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}')$.

Overview. The CKKS bootstrapping process usually proceeds in four steps, following [CHK+18a].

- $\mathsf{S2C}$ ("slots to coeffs") starts from a ciphertext $\mathsf{ct} = (a, b)$ with $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}) \approx \mathsf{Ecd}(\boldsymbol{\mu})$ for a certain message $\boldsymbol{\mu} = (\mu_j)_{0 \leqslant j < N/2} \in \mathbb{C}^{N/2}$. It returns a ciphertext $\mathsf{ct}_0 = (a_0, b_0)$ such that

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_0) \approx m' = \sum_{0 \leqslant j < N/2-1} \mathrm{Re}\left(\frac{q_0 \mu_j}{\gamma}\right) X^j + \sum_{N/2 \leqslant j < N} \mathrm{Im}\left(\frac{q_0 \mu_{j-N/2}}{\gamma}\right) X^j \ ,$$

  where $\gamma > 1$ is a gap intended to make $|\mu_j|$ small for all j;
- $\mathsf{ModRaise}$ raises the modulus of ct' from q_0 to Q, by lifting $(a_0, b_0) \in R_{q_0}^2$ to $\mathsf{ct}_{\mathrm{top}} = (a_{\mathrm{top}}, b_{\mathrm{top}})$ defined over Q_{top}. The decryption equation $a_0 s + b_0 = m' \bmod q_0$ then becomes

$$a_{\mathrm{top}} \cdot s + b_{\mathrm{top}} = m' + q_0 I \bmod Q_{\mathrm{top}} \ ,$$

  for some $I = \sum_{0 \leqslant j < N} I_j X^j \in R_{Q_{\mathrm{top}}}$.
- $\mathsf{C2S}$ ("coeffs to slots") reverts the effects of $\mathsf{S2C}$ and, on input $\mathsf{ct}_{\mathrm{top}} = (a_{\mathrm{top}}, b_{\mathrm{top}})$, returns $\mathsf{ct}_{\mathsf{EvalMod}} \in R_{Q_{\mathsf{EvalMod}}}^2$ such that

$$\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{EvalMod}}) \approx \mathsf{Ecd}\left(\left(\frac{\mu_j}{\gamma} + I_j + \iota I_{j+N/2}\right)_{0 \leqslant j < N/2}\right) \ .$$

- $\mathsf{EvalMod}$ evaluates the function $x \mapsto x \bmod 1$ slot-wise on $\mathsf{ct}_{\mathsf{EvalMod}}$ and removes the gap γ. It returns $\mathsf{ct}' \in R_{Q_{\mathrm{comp}}}^2$ such that $\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}') \approx \mathsf{Ecd}_{\mathrm{slot}}(\boldsymbol{\mu})$. In the case of a non-real cleartext vector, $\mathsf{EvalMod}$ is evaluated twice separately: once on the real part and once on the imaginary part.

Implementation of $\mathsf{C2S}$ and $\mathsf{S2C}$. $\mathsf{S2C}$ and $\mathsf{C2S}$ correspond to homomorphic (i)DFT's. As such, they can be seen either as a general matrix-vector operation which, following [CHK+18a], can be evaluated using the diagonal method [HS14]. Such an evaluation requires then $\lceil \sqrt{2N} \rceil$ homomorphic rotations, $N/2$ plaintext-ciphertext multiplications and additions, and one final rescaling operation.

In the context of CKKS bootstrapping, these operations are usually decomposed as a product of ρ sparse-diagonal linear transformations using the Cooley-Tukey decomposition [CCS19, HHC19]. This leads to an algorithm using only $O(\rho N^{1/(2\rho)})$ rotations and $O(\rho N^{1/\rho})$ plaintext-ciphertext multiplications, at the expense of a larger modulus consumption (with ρ levels).

Sparse secret encapsulation. In order to reduce failure probability (related to the $\mathsf{EvalMod}$ design) and to improve bootstrapping efficiency, an effective approach is to decrease the Hamming weight h of the secret key. This however raises security issues if done naively: evaluations keys associated to a sparse secret key compromise the secret key if they are given for a large modulus Q. It was suggested in [BTH22] to use two different keys: a sparser $\tilde{s}$ and a denser s. A key-switch is then performed at modulus q_0, from s to $\tilde{s}$, and a converse key-switch is applied just after $\mathsf{ModRaise}$. Sparse secret encapsulation ensures that Equation (1) holds for s with small Hamming weight.

Parallelization opportunities. State-of-the-art CKKS bootstrapping algorithms use RNS representation for their ciphertexts [CHK+18b]. This offers

ample opportunities for *data-level* parallelism. Using RNS, big integers modulo $Q_\ell = q_0 \dots q_\ell$ are represented as $\ell + 1$ small integers, for which most operations can be performed in a parallel way.

In contrast, opportunities for *task-level* parallelism are scarcer. S2C and C2S, which account for most of the CKKS bootstrapping computation time, include a small number (up to 15 in typical implementations for $N = 2^{16}$) of potentially parallelizable rotations. However, the use of hoisting [HS18], which extracts common operations from these rotations, significantly reduces the amount of available parallelism. Meanwhile, EvalMod is mostly sequential.

HEaaN's CKKS implementation uses the OpenMP library and takes full advantage of these parallelization opportunities.

3 SHIP: a New CKKS Bootstrapping Algorithm

In this section, we describe SHIP and study its correctness and cost. The algorithm relies on a blind rotation procedure in a black-box manner. Realizing this procedure will be the focus of Section 4.

3.1 Algorithm Description

A formal description of SHIP is provided in Algorithm 1. It takes as input a ciphertext $\mathsf{ct} = (a, b) \in R_{q_0}^2$ of the form of Equation (1): it encodes cleartexts $\mu_0, \dots, \mu_{N/2-1}$ in its coefficients. The encoding is with a gap $\gamma \gg 1$ that helps for correctness. We assume that the secret key s has low Hamming weight h: as a first approximation, the lower h, the faster SHIP. As in the sparse secret encapsulation technique [BTH22], for the sake of security, we use a larger Hamming weight for ciphertexts with modulus higher than q_0. The algorithm also takes two sorts of bootstrapping keys as inputs: blind rotation keys and masks.

The first five steps preprocess the input ciphertext $(a, b) \in R_{q_0}^2$ to view it as cleartext integers in the exponent of $\omega = \exp(2\iota\pi/q_0)$. As CKKS cleartexts are vectors of dimension $N/2$, we split the N-dimensional a and b into two halves. Note that $b_{N/2}, \dots, b_{N-1}$ do not play any role in the algorithm and are hence not converted to the root-of-unity format. Also, as we want to perform polynomial multiplication by $\mathsf{sk} = \sum_{j \in J} s_j X^j$ modulo $X^N + 1$ in the exponent and reduction modulo $X^N + 1$ may modify the signs of the coefficients, we also take the complex conjugates of the ω^{a_i}'s.

Fix $j \in J$. The aim of Steps 5 to 11 is to compute a ciphertext whose underlying cleartext vector is $(\omega^{(s_j X^j a)_i})_{0 \leqslant i < N/2}$. As shown in Lemma 1 below (see also the discussion after the lemma), this can be achieved by taking a sum of four products of pt_k for $1 \leqslant k \leqslant 4$ with well-designed masks, and then rotating the underlying cleartext vector by j positions. The multiplication by masks are of the plaintext-ciphertext type, while the specification of BlindRot is as follows:

> $\mathsf{BlindRot}(\mathsf{brkey}_j, \mathsf{ct})$: on input ct and key brkey_j corresponding to the encrypted index j, return ct' such that $\mathsf{Dcd}(\mathsf{Dec}(\mathsf{ct}')) \approx \mathsf{Rot}_j(\mathsf{Dcd}(\mathsf{Dec}(\mathsf{ct})))$.

We will provide instantiations of $\mathsf{BlindRot}$ in Section 4. Steps 5 to 11 are repeated for every $j \in J$, i.e., exactly h times. This loop produces the ciphertexts $\mathsf{ct}_\ell^{(0)}$ for $1 \leqslant \ell \leqslant h$.

Steps 13 to 21 are a binary tree multiplication of the $\mathsf{ct}_\ell^{(0)}$'s and pt_0, resulting in a ciphertext whose underlying cleartext vector is $\approx (\frac{\gamma}{4\pi}\omega^{(as)_i+b_i})_{0\leqslant i<N/2}$. Note that we have, for all $0 \leqslant i < N/2$:

$$\omega^{(as)_i+b_i} = \omega^{\lfloor \frac{q_0\mu_i}{\gamma} \rceil} \ .$$

As in [CHK+18a] and recalled in the introduction, a homomorphic conjugation (performed at Step 22) then allows to obtain a final ciphertext whose underlying cleartext vector is $\approx (\mu_i)_{0\leqslant i<N/2}$.

Algorithm 1 The SHIP (half) bootstrapping algorithm

Input: A ciphertext $(a, b) \in R_{q_0}^2$ such that $a \cdot \mathsf{sk} + b = \sum_{0\leqslant i<N/2} \lfloor \frac{q_0\mu_i}{\gamma} \rceil X^i \bmod q_0$.
Input: Bootstrapping keys $(\mathsf{brkey}_j \bmod PQ_\ell, \mathsf{Enc}_{\mathsf{pk}}(\mathsf{Ecd}(\mathbf{m}_{j,1}^{(k)} \bmod Q_\ell))_{1\leqslant k\leqslant 4})_{0\leqslant j<h}$.
Output: $(a', b') \in R_{Q'}^2$ that decrypts to $\approx \mathsf{Ecd}(\boldsymbol{\mu})$.

1: $\mathrm{pt}_0 \leftarrow \mathsf{Ecd}\left(\left(\frac{\gamma}{4\iota\pi}\omega^{b_i}\right)_{0\leqslant i<N/2}\right)$
2: $\mathrm{pt}_1 \leftarrow \mathsf{Ecd}((\omega^{a_i})_{0\leqslant i<N/2})$; $\mathrm{pt}_2 \leftarrow \mathrm{pt}_1(X^{-1})$;
3: $\mathrm{pt}_3 \leftarrow \mathsf{Ecd}((\omega^{a_i})_{N/2\leqslant i<N})$; $\mathrm{pt}_4 \leftarrow \mathrm{pt}_3(X^{-1})$
4: $\mathsf{ct}_0^{(0)} \leftarrow \mathrm{pt}_0$
5: **for** j from 0 to $h-1$ **do**
6: $\quad \mathsf{ct}_j^{(0)} \leftarrow (0, 0)$
7: $\quad$ **for** k from 1 to 4 **do** ▷ masking
8: $\quad\quad \mathsf{ct}_j^{(0)} \leftarrow \mathsf{ct}_j^{(0)} + \mathsf{PMult}(\mathrm{pt}_k, \mathsf{Enc}_{\mathsf{pk}}(\mathsf{Ecd}(\mathbf{m}_{j,1}^{(k)})))$ ▷ over $R_{Q_{\ell-1}}$
9: $\quad$ **end for**
10: $\quad \mathsf{ct}_j^{(0)} \leftarrow \mathsf{BlindRot}(\mathsf{brkey}_j, \mathsf{ct}_j^{(0)})$ ▷ blind rotation
11: **end for**
12: $n \leftarrow h + 1$
13: **for** k from 0 to $\lceil \log(h+1) \rceil - 1$ **do** ▷ product tree
14: $\quad$ **for** j from 0 to $\lfloor n/2 \rfloor - 1$ **do**
15: $\quad\quad \mathsf{ct}_j^{(k+1)} \leftarrow \mathsf{CMult}(\mathsf{ct}_{2j}^{(k)}, \mathsf{ct}_{2j+1}^{(k)})$. ▷ over $R_{Q_{\ell-2-k}}$
16: $\quad$ **end for**
17: $\quad$ **if** $n = 1 \bmod 2$ **then**
18: $\quad\quad \mathsf{ct}_{\lfloor n/2 \rfloor}^{(k+1)} \leftarrow \mathsf{ct}_{n-1}^{(k)}$
19: $\quad$ **end if**
20: $\quad n \leftarrow \lceil n/2 \rceil$
21: **end for**
22: **return** $\mathsf{ct}_0^{(\lceil \log(h+1) \rceil)} + \mathsf{Conj}(\mathsf{ct}_0^{(\lceil \log(h+1) \rceil)})$. ▷ over $R_{Q_{\ell-1-\lceil \log(h+1) \rceil}}$

The correctness of Algorithm 1 follows from the discussion above, the design of the masks (this is discussed in Subsection 3.3) and the existence of a $\mathsf{BlindRot}$ algorithm as specified above (this is discussed in Section 4). The level consumption and number of operations can be read from the algorithm. Anticipating Section 4, the cost is dominated by the blind rotations.

We present our main result as a heuristic statement, as it lacks an error analysis; we stress that apart from this aspect the proof is fully rigorous.

Theorem 1 (Heuristic). *On input* $\mathsf{ct} = (a, b) \in R_{q_0}^2$ *such that* $a \cdot \mathsf{sk} + b = \sum_{0 \leqslant i < N/2} \lfloor \frac{q_0 \mu_i}{\gamma} \rceil X^i$ *modulo* q_0*, Algorithm 1 returns a ciphertext* ct' *such that*

$$\mathsf{Dcd}(\mathsf{Dec}(\mathsf{ct}')) \approx \boldsymbol{\mu} \ ,$$

modulo $Q_{\ell-1-\lceil \log(h+1) \rceil}$*. If the secret key* sk *has Hamming weight* h*, Algorithm 1 uses* $1 + \lceil \log(h+1) \rceil$ *HE-levels and performs* $4h$ *plaintext-ciphertext multiplications,* $4h$ *ciphertext-ciphertext additions,* h *ciphertext-ciphertext multiplications,* h *calls to* $\mathsf{BlindRot}$*, and one homomorphic conjugation.*

In Section 4, we will fuse masking and $\mathsf{BlindRot}$, and describe optimizations that allow to save the HE-level consumed at Step 12 of Algorithm 1.

3.2 Remarks

We observe that Algorithm 1 maps a coefficients-encoded cleartext into a slots-encoded cleartext. Slot encoding then allows to perform CKKS computations. It is only a "half" bootstrap in that it does not explain how to convert back from slots-encoding to coefficients-encoding, in order to prepare for a subsequent bootstrapping. This can be achieved using $\mathsf{S2C}$. Using the description of Subsection 2.2, we favor the version of $\mathsf{S2C}$ consisting of a simple matrix-vector product: as it is performed at a low modulus, its cost is limited in practice; it is well parallelizable; and it consumes a single HE-level. Using a Cooley-Tukey decomposition reduces the overall number of ring operations, but it comsumes more HE-levels. Another hindrance of relying on Cooley-Tukey is that it bit-reverses the order of the slots, whereas our half bootstrap does not. After $\mathsf{S2C}$, the secret key then needs to be switched to the low-modulus secret key before the subsequent call to SHIP (as we rely on sparse secret encapsulation).

As another observation, we can assume that the most significant bits of the indices of sk corresponding to non-zero coefficients are known. This helps for blind rotation, as part of the rotation index can then be made public. Indeed, let $x_0, \ldots, x_{h-1}$ be independent uniform random variables over $[0, 1]$ and let us order them as $y_0 \leqslant y_1 \cdots \leqslant y_{h-1}$. Then, standard results on order statistics state that the residual distribution of each y_i follows the beta distribution with parameters $(i+1, h-i)$. Tail bounds for this distribution show that y_i is concentrated within $O(h^{-1/2+\varepsilon})$ of i/h with probability close to 1, for an arbitrary $\varepsilon > 0$ (see [Sko23]). In our situation, only the low-order $\log(N/\sqrt{h})$ bits of the rotation index thus need to be kept secret. In practice, we go even further, only keeping secret $\log(wN/h)$ bits for a small value of w (see Section 5.1).

Finally, we shortly explain how to adapt Algorithm 1 to the case where the values $(\mu_i)_{0 \leqslant i < N/2}$ are bits, namely $\mu_i \approx 0$ or $\mu_i \approx 1$ (as in [DMPS24, BCKS24]). In that case, we choose $\gamma = 2$ in Equation (1) so that (for all $0 \leqslant i < N/2$):

$$\frac{1 - \omega^{m_i}}{2} \approx \frac{1 - (-1)^{\mu_i}}{2} \approx \mu_i \ .$$

The above shows that, up to changing γ, Algorithm 1 also bootstraps bits. The decrease of γ leads to smaller plaintext bit-sizes inside bootstrapping, which decreases the total modulus consumption. This can be further extended to small integers encoded as in [CKKL24, BKSS24].

3.3 Masks

In this section, for a given value of j, we provide a detailed procedure for reducing the computation of $(\omega^{(a \cdot s_j X^j)_i})_{0 \leqslant i < N/2}$ to rotations by j indices. In this mathematical description, we shall disregard the fact that the rotation indices are secret and that rotations require a specific homomorphic procedure. We first focus on the underlying cleartext procedure.

We define the mask of order $0 \leqslant j \leqslant N/2$ by $\mathbf{m}_j = (\mathbf{0}_j, \mathbf{1}_{N/2-j}) \in \mathbb{R}^{N/2}$. The notation $\mathbf{0}_k$ (resp. $\mathbf{1}_k$) refers to the all-0 (resp. all-1) vector of dimension $k \geqslant 0$. For the sake of simplicity, we omit the index for $\mathbf{1}_{N/2}$. We also define $\boldsymbol{\omega}_1(\mathbf{a}) = (\omega^{a_i})_{0 \leqslant i < N/2} \in \mathbb{C}^{N/2}$ and $\boldsymbol{\omega}_2(\mathbf{a}) = (\omega^{a_{i+N/2}})_{0 \leqslant i < N/2} \in \mathbb{C}^{N/2}$, for any $\mathbf{a} \in \mathbb{Z}^N$.

Lemma 1. *Let $\mathbf{a} \in \mathbb{Z}^N$, $0 \leqslant j < N$ and $s_j \in \{-1, 1\}$. Let $\widetilde{\mathbf{a}}_j \in \mathbb{Z}^N$ be defined by $\sum_{0 \leqslant i < N} (\widetilde{\mathbf{a}}_j)_i X^i = s_j X^j \sum_{0 \leqslant i < N} a_i X^i \bmod X^N + 1$. Then we have:*

- *for $0 \leqslant j < N/2$,*

$$\begin{aligned}\boldsymbol{\omega}_1(\widetilde{\mathbf{a}}_j) = {} & \mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}_j + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}_j \\ & + \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \frac{1+s_j}{2}(\mathbf{1}-\mathbf{m}_j) + \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \frac{1-s_j}{2}(\mathbf{1}-\mathbf{m}_j) \ ;\end{aligned}$$

- *for $N/2 \leqslant j < N$, using the notation $j' = j - N/2$,*

$$\begin{aligned}\boldsymbol{\omega}_1(\widetilde{\mathbf{a}}_j) = {} & \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}_{j'} + \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}_{j'} \\ & + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \frac{1+s_j}{2}(\mathbf{1}-\mathbf{m}_{j'}) + \mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \frac{1-s_j}{2}(\mathbf{1}-\mathbf{m}_{j'}) \ .\end{aligned}$$

The proof, provided in the extended version of this paper, proceeds by algebraic inspection. The identities can also be tested using the SAGE code provided as supplementary material. Similar formulas exist for the $\boldsymbol{\omega}_2(\widetilde{\mathbf{a}}_j)$'s, but we do not need them in SHIP (see appendix). In the sequel, for all $0 \leqslant j < N$, we define masks $\mathbf{m}_{j,1}^{(k)}$ for $1 \leqslant k \leqslant 4$ such that:

$$\begin{aligned}\boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j) = {} & \mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \mathbf{m}_{j,1}^{(1)} + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \mathbf{m}_{j,1}^{(2)} \\ & + \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \mathbf{m}_{j,1}^{(3)} + \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \mathbf{m}_{j,1}^{(4)} \ .\end{aligned}$$

The existence and values of the masks can be derived from Lemma 1. Note that the masks $\mathbf{m}_{j,1}^{(k)}$ and $\mathbf{m}_{j,2}^{(k)}$ for $1 \leqslant k \leqslant 4$ depend on s_j. These are the masks used in Algorithm 1. As a side note, we observe that $\sum_{1 \leqslant k \leqslant 4} \mathbf{m}_{j,\delta}^{\prime(k)} = \sum_{1 \leqslant k \leqslant 4} \mathbf{m}_{j,\delta}^{(k)} = 1$

holds for $\delta \in \{1, 2\}$. This can be used for reducing the total memory occupation of the masks.

To obtain a more compact form, we define $\boldsymbol{\pi}_1(\mathbf{a}) = \boldsymbol{\omega}_1(\mathbf{a})$, $\boldsymbol{\pi}_2(\mathbf{a}) = \boldsymbol{\omega}_1(-\mathbf{a})$, $\boldsymbol{\pi}_3(\mathbf{a}) = \boldsymbol{\omega}_2(\mathbf{a})$ and $\boldsymbol{\pi}_4(\mathbf{a}) = \boldsymbol{\omega}_2(-\mathbf{a})$, so that the equations above can be rewritten as follows, for $\delta \in \{1, 2\}$:

$$\boldsymbol{\omega}_\delta(\tilde{\mathbf{a}}_j) = \sum_{1 \leqslant k \leqslant 4} \mathsf{Rot}_j(\boldsymbol{\pi}_k(\mathbf{a})) \odot \mathbf{m}_{j,\delta}^{(k)} \quad . \tag{4}$$

Rotations being costly to evaluate homomorphically, when possible, it may be considered to "factor out" Rot_j. This leads to the following equivalent equation, for $b \in \{1, 2\}$:

$$\boldsymbol{\omega}_\delta(\tilde{\mathbf{a}}_j) = \mathsf{Rot}_j \left(\sum_{1 \leqslant k \leqslant 4} \boldsymbol{\pi}_k(\mathbf{a}) \odot \mathbf{m}_{j,\delta}^{\prime(k)} \right) \quad , \tag{5}$$

where $\mathbf{m}_{j,\delta}^{\prime(k)} = \mathsf{Rot}_{-j}(\mathbf{m}_{j,\delta}^{(k)})$.

4 Blind Rotations

Consider Equations (4) or (5). We now aim at evaluating them homomorphically in the context of bootstrapping: the vector $\mathbf{a}$ is known in clear (it is a component of a ciphertext, but its meaning does not matter here), whereas the secret key sk is provided in an encrypted form that is yet to be determined. This precise form should be designed to facilitate the homomorphic evaluation of Equations (4) or (5). Those equations depend on sk in two ways: the masks $\mathbf{m}_{j,\delta}^{\prime(i)}$ and the rotation index j. Given sk, key generation builds the required masks, encoded in slots and encrypted. Encrypting j such that $s_j \neq 0$ should be done in such a way as to enable the evaluation of Rot_j, namely a rotation by a secret index. In this section, we first consider two approaches to instantiate blind rotations, then we compare them and propose a hybrid approach.

4.1 Column Method

Assume that the secret index j is known to belong to a certain subset $I \subseteq [0, N-1]$. As part of the bootstrapping key, we provide the ciphertexts

$$\mathsf{cmkey}^{(j)} = \left(\varepsilon_i^{(j)}\right)_{i \in I} = (\mathsf{Enc}_{\mathsf{pk}}(\mathsf{Ecd}(\mathbf{1}_{i=j})))_{i \in I} \quad .$$

In $\mathsf{cmkey}^{(j)}$, there is exactly one ciphertext per element of $i \in I$, which encodes the scalar 0 in all slots if $j \neq i$ and the scalar 1 in all slots if $j = i$. We can then realize $\mathsf{BlindRot}$ as follows:

$$\forall j \in I, \boldsymbol{\mu} \in \mathbb{C}^{N/2} : \ \mathsf{BRotCM}\left(\mathsf{cmkey}^{(j)}, \boldsymbol{\mu}\right) := \sum_{i \in I} \mathsf{Ecd}\left(\mathsf{Rot}_i(\boldsymbol{\mu})\right) \cdot \varepsilon_i^{(j)} \quad . \tag{6}$$

The input $\mathsf{cmkey}^{(j)}$ is the encrypted rotation index, and the input $\boldsymbol{\mu} \in \mathbb{C}^{N/2}$ is a cleartext message. If we stack the column vectors $\mathsf{Rot}_i(\boldsymbol{\mu})$ into a matrix, then the above $\mathsf{BlindRot}$ realization can be viewed as the plaintext-ciphertext variant of the so-called column method matrix-vector multiplication [HS14]. Overall, for any $j \in I$ and any $\boldsymbol{\mu} \in \mathbb{C}^{N/2}$, we have

$$\mathsf{Dcd}\left(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{BRotCM}(\mathsf{cmkey}^{(j)}, \boldsymbol{\mu}))\right) \approx \mathsf{Rot}_j(\boldsymbol{\mu}) \ .$$

Modulus management. The result of the computation corresponding to Equation (6) must be rescaled before further homomorphic computation. Such a rescaling incurs a modulus loss equal to the scaling factor. In the present situation, we can avoid this modulus loss. Recall that CKKS decomposes its total modulus budget under the form PQ, where P is an auxiliary modulus used for key-switching operations (rotations, conjugations, relinearization within homomorphic ciphertext-ciphertext multiplication), whereas Q is the "computational" modulus. Here, we assume that the ciphertexts in the $\mathsf{cmkey}^{(j)}$'s are provided modulo PQ, and evaluate Equation (6) as described in Algorithm 2. Equation (6) does not involve any keyswitching, so the auxiliary modulus P is not required to evaluate it. We can thus encode $\boldsymbol{\mu}$ at the maximal modulus PQ, and use P as a scaling factor for the elements of $\varepsilon_k^{(j)}$. After rescaling by P, we will be back to the usual situation of a ciphertext with modulus Q.

Algorithm 2 Blind rotation based on the column method

Input: A cleartext $\boldsymbol{\mu} \in \mathbb{C}^{N/2}$.
Input: Key $\mathsf{cmkey}^{(j)} = (\varepsilon_i^{(j)})_{i \in I}$ over R_{PQ}, encoded with scaling factor P.
Output: A ciphertext modulo Q that decrypts to $\approx \mathsf{Rot}_j(\boldsymbol{\mu})$.

```
ct ← (0,0) ∈ R_PQ^2
pt ← Ecd(μ) ∈ R_PQ
for i in I do
    pt_rot ← pt(X^{5^i}) ∈ R_PQ
    ct = ct + pt_rot · ε_i^(j) ∈ R_PQ^2
end for
return Rescale_P(ct) ∈ R_Q^2
```

We now analyze the cost of Algorithm 2, as a function of $|I|$. The bootstrapping key component $\mathsf{cmkey}(j)$ contains $|I|$ ciphertexts; evaluating a blind rotation costs $|I|$ cleartext rotations, $|I|$ plaintext-ciphertext multiplications and $|I| - 1$ ciphertext-ciphertext additions. These are performed without rescaling, by rescaling only once at the end.

4.2 Mux Method

We now describe a second method for performing blind rotations. This method is based on the use of a Mux-type operator, which depends on an encrypted bit that is part of the bootstrapping key.

Homomorphic half-mux-rotate. We first describe the $\mathsf{HMuxRot}$ operator, which is tailored to our needs. It is a fused Mux and Rot: it takes as arguments a bit β and a rotation index j and, on input ct, returns either an encryption of 0 (if $\beta = 0$) or $\mathsf{Rot}(\mathsf{ct})$ (if $\beta = 1$). It is only a 'half-mux' as it returns 0 if the selector bit is 0.

Definition 1. *Let $\mathsf{sk} \in R$ be a secret key, Q be the ciphertext modulus and P be the auxiliary modulus. For $0 \leqslant j < N/2$, the $\mathsf{HMuxRot}$ key $\mathsf{hmrkey}_\beta^{(j)}$ of the test bit $\beta \in \{0,1\}$ consists in two ciphertexts $\mathsf{hmrkey}_{\beta,0}^{(j)} = (a_0, b_0)$ and $\mathsf{hmrkey}_{\beta,1}^{(j)} = (a_1, b_1)$ over R_{PQ} such that*

$$\begin{aligned} a_0 \cdot \mathsf{sk} + b_0 &= P\beta \cdot \mathsf{sk}(X^{5^j}) + \varepsilon_0 \bmod PQ \ , \\ a_1 \cdot \mathsf{sk} + b_1 &= P\beta + \varepsilon_1 \bmod PQ \ , \end{aligned}$$

where ε_0 and ε_1 are small elements of R.

For $\mathsf{ct} = (a, b) \in R_Q^2$, we define

$$\begin{aligned} &\mathsf{HMuxRot}^{(j)}\left(\mathsf{hmrkey}_\beta^{(j)}, \mathsf{ct}\right) \\ &\qquad = \left\lfloor P^{-1}\left(a' \cdot \mathsf{hmrkey}_{\beta,0}^{(j)} + b' \cdot \mathsf{hmrkey}_{\beta,1}^{(j)} \bmod PQ\right)\right\rceil \in R_Q^2 \ , \end{aligned}$$

where $a' = a(X^{5^j})$ and $b' = b(X^{5^j})$ are viewed as elements of R_{PQ}.

The rotation is only used as a key-switch (from sk to $\mathsf{sk}(X^{5^j})$). We can similarly combine Mux with any homomorphic operation based on key-switching, such as conjugation.

Lemma 2. *We use the notations from Definition 1. Assume that $P \geqslant Q$ and that $\mathsf{sk} \in R$ is small. Let $\boldsymbol{\mu} \in \mathbb{C}^{N/2}$ and assume that the ciphertext $\mathsf{ct} = (a, b) \in R_Q^2$ satisfies $a \cdot \mathsf{sk} + b \approx \mathsf{Ecd}(\boldsymbol{\mu}) \bmod Q$. Then, we have:*

$$\mathsf{Dec}_{\mathsf{sk}}\left(\mathsf{HMuxRot}^{(j)}\left(\mathsf{hmrkey}_\beta^{(j)}, \mathsf{ct}\right)\right) \approx \beta \cdot \mathsf{Ecd}\left(\mathsf{Rot}_j(\boldsymbol{\mu})\right) \ .$$

If $\beta = 0$, then the underlying cleartext becomes the all-zero vector, whereas if $\beta = 1$, it becomes a rotation of $\boldsymbol{\mu}$ by j indices. As β is encrypted, the evaluator does not know which case holds. The proof is provided in the full version of this paper.

Mux method for blind-rotate. With $\mathsf{HMuxRot}$ at hand, we can implement a blind rotation of any index $0 \leqslant j < N$ by considering its binary decomposition $j = \sum_{0 \leqslant k < n} j_k 2^k$ with $j_k \in \{0,1\}$ for all k and $n = \log N$. The strategy is then to rotate by 2^k indices if $j_k = 1$ and to keep the underlying cleartext if $j_k = 0$, repeatedly over all values of k (in any order). Recall that $\mathsf{HMuxRot}$ indeed rotates by the underlying cleartext by the appropriate amount if the test bit is equal to 1, but sends it to the all-0 vector otherwise. To handle this, we add the outcomes of two $\mathsf{HMuxRot}$'s for every k: a first one that rotates by 2^k

indices if the test bit j_k is equal to 1, and a second one that rotates by 0 if the test bit $1 - j_k$ is equal to 1. Note that combining two such $\mathsf{HMuxRot}$'s gives a 'full' MuxRot gate. Overall, this leads to providing the following mux-method key as part of the bootstrapping key:

$$\mathsf{mmkey}^{(j)} := \left(\mathsf{hmrkey}_{j_k}^{(2^k)}, \mathsf{hmrkey}_{1-j_k}^{(0)}\right)_{0 \leqslant k < n} .$$

Algorithm 3 described the Mux-based blind-rotate.

Algorithm 3 $\mathsf{BRotMux}$: Blind rotation based on the mux method

Input: A ciphertext ct.
Input: All $\mathsf{hmrkey}_{j_k}^{(2^k)}$'s and $\mathsf{hmrkey}_{1-j_k}^{(0)}$'s from $\mathsf{mmkey}^{(j)}$.
Output: A ciphertext that decrypts to $\approx \mathsf{Rot}_j(\boldsymbol{\mu})$, where $\boldsymbol{\mu} = \mathsf{Dcd}(\mathsf{Dec}(\mathsf{ct}))$.

```
ct_out ← ct
for k = 0 to n − 1 do
    ct_0 ← HMuxRot^(2^k)(hmrkey_{j_k}^(2^k), ct_out)
    ct_1 ← HMuxRot^(0)(hmrkey_{1−j_k}^(0), ct_out)
    ct_out ← ct_0 + ct_1
end for
return ct_out
```

Lemma 3. *On inputs* $\mathsf{mmkey}^{(j)}$ *and* ct *such that* $\mathsf{Dcd}(\mathsf{Dec}(\mathsf{ct})) = \boldsymbol{\mu}$*, Algorithm 3 returns* ct' *such that* $\mathsf{Dcd}(\mathsf{Dec}(\mathsf{ct}')) \approx \mathsf{Rot}_k(\boldsymbol{\mu})$*, using* $2 \log N$ $\mathsf{HMuxRot}$ *keys and making* $2 \log N$ *calls to* $\mathsf{HMuxRot}$*.*

The result follows from Lemma 2 and inspection of Algorithm 3.

4.3 Comparing and Combining the Two Methods

The column method relies on homomorphic plaintext-ciphertext multiplications and ciphertext-ciphertext additions, which are the most efficient native CKKS operations. It also enjoys high-grain parallelism. On the downside, its cost grows linearly in the size of the range of the encrypted index j. Oppositely, the mux method involves more costly keyswitching operations and is more sequential. Its main advantage is that its cost grows only logarithmically in the size of the range of the encrypted index j.

Note that neither method consumes any multiplicative level: no homomorphic multiplication is involved in the mux method, and the auxiliary prime P is exploited in the column method to scale plaintexts so as to avoid rescaling ciphertexts below the input modulus Q.

Finally, the methods have different input formats. In the column method, the cleartext vector to be rotated should be provided unencrypted. In the mux method it can be encrypted, and hence unencrypted as well.

To exploit the respective advantages of both methods, we combine them in a baby-step giant-step spirit. As the column method requires cleartext inputs, when combining the two methods, the column method needs to be used before the mux method. Given a rotation index $0 \leqslant j < N$, and given a threshold $\theta \in (0, N)$, we write $j = j_0 + j_1\theta$ with $0 \leqslant j_0 < \theta$ and proceed as follows:

- compute $\mathsf{ct} = \mathsf{BRotCM}(\mathsf{cmkey}^{(j_1 \cdot \theta)}, \mathsf{Ecd}(\boldsymbol{\mu}))$;
- compute and return $\mathsf{BRotMux}(\mathsf{mmkey}^{(j_0)}, \mathsf{ct})$.

Regarding the first step, we observe that the column method has to pick among N/θ possible values; the cost of the first step is thus of the order of N/θ elementary homomorphic operations. The second step involves $O(\log \theta)$ calls to $\mathsf{HMuxRot}$. The optimal choice of θ must be determined experimentally, and depends on the difference of cost between basic elementary homomorphic operations (such as plaintext-ciphertext multiplications) and more advanced ones (based on key-switching).

4.4 Fusing Blind Rotations and Masks

Fusing the column method with masks. We observe that, in our context, the column method can be fused with Equation (4) to decrease modulus consumption, compared to evaluating Equation (4) and then multiplying by a mask. Using the cleartext equation underlying Equation (6) to re-express $\mathbf{m}_{j,1}^{(k)}$ in Equation (4), we obtain, for $0 \leqslant j < N$:

$$\begin{aligned} \boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j) &= \sum_{i \in I} \sum_{1 \leqslant k \leqslant 4} \mathbf{1}_{i=j} \odot \mathsf{Rot}_i(\boldsymbol{\pi}_k(\mathbf{a})) \odot \mathbf{m}_{j,1}^{(k)} \\ &= \sum_{i \in I} \sum_{1 \leqslant k \leqslant 4} \mathsf{Rot}_i(\boldsymbol{\pi}_k(a)) \odot \left(\mathbf{1}_{i=j} \odot \mathbf{m}_{j,1}^{(k)} \right) \quad . \end{aligned} \tag{7}$$

By providing encryptions of $\mathbf{M}^{(k)}(j) = (\mathbf{1}_{i=j} \odot \mathbf{m}_{j,1}^{(k)})_{i \in I}$ for $1 \leqslant k \leqslant 4$ as a part of the bootstrapping key, we can thus compute $\boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j)$ with no modulus usage as the auxiliary modulus can be used for the plaintext-ciphertext multiplications involved in Equation (7) (see Subsection 4.1).

Fusing the mixed method with masks. We note that in the context of the column method it does not seem interesting to rely on Equation (5) rather than Equation (4). Indeed, in that case Equation (7) becomes:

$$\boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j) = \sum_{i \in I} \mathbf{1}_{i=j} \odot \mathsf{Rot}_i \left(\sum_{1 \leqslant k \leqslant 4} \boldsymbol{\pi}_k(a) \odot \mathbf{m}_{j,1}^{\prime(k)} \right) \quad ,$$

involving costly ciphertext-ciphertext multiplications and homomorphic rotations while making modulus saving more delicate. However, in the context of the mixed method, one can move from Equation (4) to Equation (5) between

the two rotation steps, using the following identity (with $j = j_1\theta + j_0$):

$$\boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j) = \mathsf{Rot}_{j_0}\left(\sum_{1\leqslant k\leqslant 4} \mathsf{Rot}_{j_1\theta}(\boldsymbol{\pi}_k(\mathbf{a})) \odot \mathsf{Rot}_{-j_0}\left(\mathbf{m}_{j,1}^{(k)}\right)\right) .$$

In the context of a blind rotation, the internal rotation call is based on the column method and computed as

$$\sum_{\substack{0\leqslant j_1\leqslant \lfloor (N-1)/\theta\rfloor \\ 1\leqslant k\leqslant 4}} \mathsf{Rot}_{j_1\theta}(\boldsymbol{\pi}_k(\mathbf{a})) \odot \left(\mathbf{1}_{j_1=\lfloor \frac{j}{\theta}\rfloor} \odot \mathsf{Rot}_{-j_0}\left(\mathbf{m}_{j,1}^{(k)}\right)\right) , \tag{8}$$

whereas the external Rot call uses the mux method. In this context, the column method key is $\mathbf{M}^{(k)}(j) = (\mathbf{1}_{j_1=\lfloor j/\theta\rfloor} \odot \mathsf{Rot}_{-j_0}(\mathbf{m}_{j,1}^{(k)}))_{0\leqslant j_1\leqslant \lfloor (N-1)/\theta\rfloor}$.

Algorithm 4 $\mathsf{BRotHybrid}$: Blind rotation based on the hybrid method

Input: $(\mathrm{pt}_i)_{1\leqslant i\leqslant 4} \in R_{PQ}$ as in Steps 2-3 of Algorithm 1.
Input: Keys $\mathbf{M}^{(k)}(j) = \mathsf{Enc}_{\mathsf{pk}}(\mathsf{Ecd}(\mathbf{1}_{j_1=\lfloor \frac{j}{\theta}\rfloor} \odot \mathsf{Rot}_{(-j \bmod \theta)}(\mathbf{m}_{j,1}^{(k)})))_{0\leqslant j_1\leqslant \lfloor \frac{N-1}{\theta}\rfloor}$ over R_{PQ}.
Input: Bootstrapping keys $\mathsf{mmkey}^{(j \bmod \theta)}$.
Output: $\mathsf{ct} \in R_Q^2$ which decrypts to $\approx \mathsf{Ecd}\left((\omega^{(a\cdot s_j X^j)_i})_{0\leqslant i<N/2}\right)$.

1: $\mathsf{ct} \leftarrow (0,0)$
2: **for** $k = 1$ **to** 4 **do**
3: **for** $j_1 = 0$ **to** $\lfloor (N-1)/\theta\rfloor$ **do**
4: $\mathsf{ct} \leftarrow \mathsf{ct} + \mathsf{PCMult}(\mathrm{pt}_k(X^{5^{j_1\theta}}), (\mathbf{M}^{(k)}(j))_{j_1})$
5: **end for**
6: **end for**
7: $\mathsf{ct} \leftarrow \mathsf{Rescale}_P(\mathsf{ct})$ $\triangleright$ $\mathsf{ct} \in R_Q^2$.
8: **return** $\mathsf{BRotMux}_{\mathsf{mmkey}^{(j \bmod \theta)}}(\mathsf{ct})$

Algorithm 4, based on Equation (8), is our complete procedure for computing blind rotations. This algorithm is meant to replace Steps 6-10 of Algorithm 1; in order to use the modulus management trick described at the end of Subsection 4.1, Steps 1-3 of Algorithm 1 should also be performed over R_{PQ_ℓ} rather than over R_{Q_ℓ}.

We note that we can also use BRotCM for the rotation of index j_0, followed by $\mathsf{BRotMux}$ for the rotation of index $j_1\theta$. This however requires adapting Algorithm 3 to handle the information that the rotation index is known to be divisible by θ. For the sake of simplicity, we chose to describe the former version, even though our implementation uses the latter (see Section 5.4).

5 Implementation and Experiments

Before describing our implementation and experimental results, we first give algorithmic optimizations that prove effective in practice.

5.1 Algorithmic Optimizations

Gadget decomposition in $\mathsf{HMuxRot}$. The RNS gadget decomposition technique [BEHZ16] allows one to weaken the assumption $P \geqslant Q$; we outline how this procedure can be adapted to our setting, and refer the reader to [BEHZ16] for full details. Fix an integer dnum such that Q can be written as $Q = D_1 \dots D_{\mathsf{dnum}}$ with $\max_i D_i \leqslant P$. For $i \leqslant r$, we define $\gamma_i = ((D/D_i)^{-1} \bmod D_i) \cdot (D/D_i)$, so that $\gamma_i = 1 \bmod D_i$ while $\gamma_i = 0 \bmod D_j$ for $j \neq i$. In this setting, we can define the $\mathsf{HMuxRot}$ key $\mathsf{hmrkey}_\beta^{(j)}$ as $2 \cdot \mathsf{dnum}$ ciphertexts (a_{i0}, b_{i0}) and (a_{i1}, b_{i1}) over R_{PQ} that satisfy, for all i:

$$a_{i0} \cdot \mathsf{sk} + b_{i0} \approx P\beta\gamma_i \cdot \mathsf{sk}(X^{5^j}) \bmod PQ \ ,$$
$$a_{i1} \cdot \mathsf{sk} + b_{i1} \approx P\beta\gamma_i \bmod PQ \ .$$

Mux-based blind rotate can then be implemented with Algorithm 5; in this algorithm, for any two odd integers $p|q$, $\mathrm{Lift}_{p,q}$ is the map from R_p^2 to R_q^2 that sends each coefficient modulo p to $\{-(p-1)/2, \dots, (p-1)/2\} \subseteq \mathbb{Z}/q\mathbb{Z}$. For $\mathsf{dnum} = 1$, we recover the $\mathsf{HMuxRot}$ formula from Definition 1.

Algorithm 5 $\mathsf{HMuxRot}$ with gadget decomposition

Input: A ciphertext $\mathsf{ct} = (\alpha, \beta) \in R_Q^2$.
Input: All (a_{i0}, b_{i0})'s and (a_{i1}, b_{i1})'s of $\mathsf{hmrkey}_\beta^{(j)}$.
Output: A ciphertext modulo Q that decrypts to $\approx \beta \cdot \mathsf{Ecd}(\mathsf{Rot}_j(\boldsymbol{\mu}))$.

```
ct_out ← (0,0) ∈ R_PQ^2
for i from 1 to dnum do
    (α_i, β_i) ← Lift_{D_i,PQ}((α, β) mod D_i) mod PQ
    (α_rot,i, β_rot,i) ← (α_i(X^{5^j}), β_i(X^{5^j}))
    ct_out ← ct_out + α_rot,i · (a_i0, b_i0) + β_rot,i · (a_i1, b_i1) mod PQ
end for
return ⌊P^{-1} ct_out⌉ ∈ R_Q^2.
```

The same computations as in the proof of Lemma 2 ensure the correctness of Algorithm 5, except that the decryption error term is now small even if P only satisfies the weaker condition $P \geqslant \max D_i$. Algorithm 5 uses the hoisting techniques from [HS18] and [BMTH21].

B-to-1 mux-rotate. From the $\mathsf{HMuxRot}$ gate, we construct a $|I|$-way rotate and select which, on input a set of keys $\mathsf{hmrkey}_{\beta_j}^{(j)}$ for $j \in I = \{j_0, \dots, j_{B-1}\}$ and a ciphertext ct such that $\mathsf{Dcd}(\mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct})) = \boldsymbol{\mu}$, returns

$$\sum_{j \in I} \beta_j \cdot \mathsf{Rot}_j(\boldsymbol{\mu}) \ .$$

This is achieved by computing

$$\sum_{j \in I} \mathsf{HMuxRot}^{(j)}(\mathsf{hmrkey}_{\beta_j}^{(j)}, \mathsf{ct}) \ .$$

This B-to-1 mux-rotate can be used in Algorithm 3 to replace the binary decomposition of the rotation index by a base B-decomposition. For a rotation index $\leqslant W$, this brings the asymptotic number of calls of $\mathsf{HMuxRot}$ to $B\lceil(\log W)/\log B\rceil$, which suggests that the optimal B is very small. However, using hoisting as in [HS18, BMTH21] makes $B \geqslant 3$ often better in practice than using $B = 2$.

Regularly-spaced nonzero secret key coefficients. We make the assumption that, for $\mathsf{sk} = \sum_{0 \leqslant k < h} \mathsf{sk}_{i_k} X^{i_k} \in R$, the index i_k of the k-th nonzero coefficient is in $[kN/h - w, kN/h + w)$, for some integer w. This enables a reduction of the blind rotations cost, as the unknown rotation index belongs to an interval of size $2w$ instead of N. This slightly reduces the total entropy of the key; to the best of our knowledge, the best attack applicable in that context was proposed in [May21], and gives a complexity of the order of $(4w)^{31 \cdot 0.4}$ scalar products between vectors of dimension N. For $N \geqslant 2^{13}$, we obtain 128 bits of security as soon as $(4w)^{31 \cdot 0.4} > 2^{115}$. We used $w = \max(175, N/(2h))$ in our experiments.

5.2 Implementation Remarks

$\mathsf{S2C}$. Our implementation does not use the Cooley-Tukey decomposition for $\mathsf{S2C}$ as in [HHC19]. As pointed in Section 2.2, our algorithm would then return a bootstrapped ciphertext with bit-reversed slot-order. We thus revert to the original description of [CHK+18a], where $\mathsf{S2C}$ is performed as a plaintext matrix / ciphertext vector product, using the diagonal method with baby-step giant-step optimization [HS14].

Parameter choices. The computational cost of our algorithm is directly connected to h, the Hamming weight of the secret key at modulus q_0. In our implementation, we have used $h = 31$ and evaluated the security using the lattice estimator [APS15] at modulus $P \cdot q_0$ (as we rely on a switching key modulo $P \cdot q_0$). The latter guarantees 128-bit security for the values used in our experiments, namely $(\log N, \log(Pq_0)) \in \{(13, 55), (14, 100), (15, 105)\}$ (see Section 5.4).

We mainly used four parameter sets for the experiments: LL13, LL14, HT14, and HT15, which are shown in Table 2. LL represents a low-latency parameter set with one multiplicative level, and HT represents a high-throughput parameter set with many multiplicative levels. While LL13 and HT14 are designed for a relatively low precision values (around 4 bits), LL14 and HT15 are designed for higher precision values over 16 bits.

Each parameter meets the standard 128-bit security constraints. We used a dense ternary secret for encryption and switched to a sparse secret with $h = 31$ at the lowest level (after $\mathsf{S2C}$) based on [BTH22].

5.3 Comparison with HEaaN

Achieving a fair comparison between SHIP and HEaaN raises two issues. On the one hand, SHIP uses sparse secret encapsulation [BTH22], whereas HEaaN does not. On the other hand, HEaaN bootstrapping has a significant failure

Table 2. Parameter sets used for the experiments. θ is the size of each column-method-based blind rotation and ℓ_0/θ is the size of each mux-method-based blind rotation, where B is the base for its decomposition.

Scheme	Param. set	N	$\log PQ$	dnum	(Avg. prime bits) × (# of primes)					BlindRot		
					Base	S2C	Mult	Boot	Aux	θ	ℓ_0/θ	B
SHIP	LL13	2^{13}	218	8	25 × 1	20 × 1	20 × 1	25 × 5	28 × 1	6	64	4
	LL14	2^{14}	437	4	48 × 1	37 × 1	37 × 1	43 × 5	50 × 2	9	64	4
	HT14	2^{14}	438	8	28 × 1	23 × 1	23 × 9	25 × 5	29 × 2	9	64	4
	HT15	2^{15}	881	4	50 × 1	39 × 1	39 × 9	46 × 5	53 × 4	17	64	4
CKKS (HEaaN)	Param15	2^{15}	771	16	49 × 1	33 × 3	36 × 1	48 × 11	54 × 1	-		
	Param16	2^{16}	1555	5	58 × 1	42 × 3	42 × 9	58 × 12	59 × 5	-		

probability due to a limited approximation range in EvalMod, while SHIP does not. Using sparse secret encapsulation [BTH22] in HEaaN would not actually bring an efficiency improvement, but would solve the second issue for $N = 2^{16}$. For $N = 2^{15}$ or $N = 2^{14}$ for bootstrapping bits, fixing the second issue requires dedicating one further level to EvalMod. It should not incur any significant efficiency penalty, but would reduce the number of multiplicative levels by 1. Despite those differences, we chose to compare ourselves with HEaaN, which is the reference implementation for CKKS bootstrapping. We have corrected the throughput corresponding to $N = 2^{14}$ in Table 5 to account for the reduction in the number of levels. We did not do the same for $N = 2^{15}$ as it only enjoys one multiplicative level.

5.4 Experiments

We implemented SHIP based on the HEaaN library [Cry22] with AVX2 support and used OpenMP for multi-threading. All the experiments were performed on a two-socket system with two AMD EPYC 7473X CPU chips and 512GB of DDR4-3200 DRAM. Each CPU chip has 24 cores and runs at 2.8GHz (3.7GHz for the boost clock). We turned off the simultaneous multi-threading (SMT) feature from the CPUs and controlled the number of used CPU cores using numactl in the experiments.

We measured the latency and precision of boostrapping while varying the parameter set and the number of utilized CPU cores. We took median values for 100 trials. For bootstrapping real numbers, precision is measured as $\log_2 \epsilon^{-1}$ (bits), where ϵ is the maximum magnitude of real error among the slots. To compare the throughput, we used the following metric, which incorporates the number of total bits refreshed by bootstrapping as well as its latency and frequency.

$$\text{Throughput} = \frac{(\text{\# of slots}) \times (\text{\# of mult levels}) \times (\text{Precision})}{(\text{Latency})} \tag{9}$$

Lightweight bootstrapping for real numbers. We used the parameter sets provided in the HEaaN library (see Table 2) to compare SHIP with the baseline CKKS boostrapping for real numbers. The results are shown in Table 3. While

Table 3. Experimental results of performing bootstrapping for real numbers using SHIP or the baseline bootstrapping method in the HEaaN library. The column "# of mult levels" refers to the levels outside of bootstrapping. We show relative throughput compared to that of the single-core (1-core) LL13 case. We highlight best-performing cases of SHIP in boldface.

Scheme	Param. set	# of slots ($N/2$)	# of mult levels	Precision (bits)	Latency		Throughput	
					1-core	32-core	1-core	32-core
SHIP	LL13	2^{12}	1	4.45	**3.02s**	**215ms**	1×	14.1×
	LL14	2^{13}	1	16.9	4.91s	333ms	4.68×	69.0×
	HT14	2^{13}	9	4.55	13.2s	1.09s	4.22×	51.0×
	HT15	2^{14}	9	18.1	29.6s	1.63s	**15.0**×	**271**×
CKKS (HEaaN)	Param15	2^{14}	1	16.7	3.44s	624ms	13.2×	72.7×
	Param16	2^{15}	9	22.0	8.68s	1.09s	124×	989×

SHIP can be performed with a value of N as small as 2^{13}, the baseline bootstrapping requires a much larger N. For Param15 and Param16, the baseline bootstrapping uses 14–15 HE-levels, which leads to use a high $\log PQ$, and thus a high degree N. While using a large N is beneficial for throughput as can be seen from Table 3, it results in higher latency.

Given these trade-offs, SHIP enables a lightweight instantiation of boostrapping real numbers with latency as low as 215ms. Furthermore, while the version of the HEaaN library that we used exclusively uses 64-bit word-size, it is possible to either use a smaller 32-bit word-size for LL13 and HT14, or to replace RNS-CKKS with the new design for the moduli system from [CCKK24]. Both options would open further latency improvement opportunities.

Alternatively, by using a larger N value (e.g., HT15 with $N = 2^{15}$), we could achieve comparable throughput to the baseline bootstrapping method. For the same $N = 2^{15}$, HT15 shows 1.14× and 3.72× higher throughput compared to Param15 in single-core and 32-core executions. The increased parallelism in SHIP enables faster execution for multi-core environments. Yet, Param16 with a larger N value shows much higher throughput compared to HT15 and Param15.

Multi-core scalability. Due to the additional parallelization opportunities, SHIP exhibits a good multi-core scalability. Especially, as we can exploit the parallelism of $h = 31$ independent loops, high speedups of 12.1–18.1× were achieved for up to 32 cores. By contrast, the speedups for the baseline CKKS bootstrapping method only reaches 5.18–8.16× even when utilizing all 48 cores in our system.

For the baseline method, Param15 especially shows limited multi-core scalability because the degree of parallelism decreases as the overall data size decreases for smaller parameters. SHIP is free from these problems as the main source of parallelism is not dependent on the data size but on the computational flow ($h = 31$ independent loops).

Adjusting blind rotation parameters. To analyze the impact of choosing different blind rotation parameters, we measured the single-core bootstrapping latency of LL13 for various parameters (see Fig. 1). Mixing the use of column

Table 4. Latency and speedup of bootstrapping for real numbers using SHIP or the baseline bootstrapping method in the HEaaN library, depending on the number of utilized CPU cores.

Scheme	Param. set	Latency (speedup)						
		1-core	2-core	4-core	8-core	16-core	32-core	48-core
SHIP	LL13	3.02s	1.59s	803ms	446ms	290ms	215ms	239ms
		(1×)	(1.90×)	(3.76×)	(6.78×)	(10.4×)	(14.1×)	(12.6×)
	LL14	4.91s	2.57s	1.29s	696ms	418ms	333ms	342ms
		(1×)	(1.91×)	(3.80×)	(7.06×)	(11.7×)	(14.7×)	(14.4×)
	HT14	13.2s	7.30s	3.62s	2.01s	1.49s	1.09s	915ms
		(1×)	(1.80×)	(3.64×)	(6.54×)	(8.87×)	(12.1×)	(14.4×)
	HT15	29.6s	12.1s	6.09s	3.30s	2.00s	1.63s	1.62s
		(1×)	(2.45×)	(4.85×)	(8.97×)	(14.8×)	(18.1×)	(18.3×)
CKKS (HEaaN)	Param15	3.44s	2.12s	1.20s	903ms	644ms	624ms	665ms
		(1×)	(1.63×)	(2.87×)	(3.81×)	(5.35×)	(5.52×)	(5.18×)
	Param16	8.68s	4.80s	2.74s	1.90s	1.35s	1.09s	1.06s
		(1×)	(1.81×)	(3.17×)	(4.58×)	(6.43×)	(7.97×)	(8.16×)

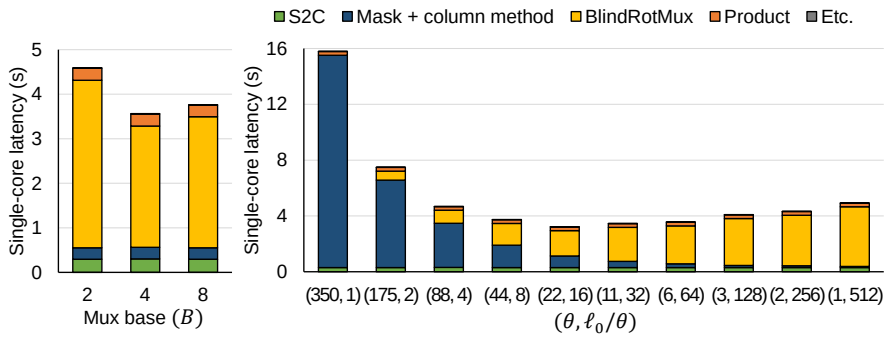

Fig. 1. Single-core LL13 bootstrapping latency when $(\theta, \ell_0/\theta) = (6, 64)$ and the mux base B varies from 2 to 8 (left) and when $(\theta, \ell_0/\theta)$ ($\ell_0 \geqslant 350$) varies from $(350, 1)$ to $(1, 512)$ and the mux base $B = 4$ (right).

method and mux method is crucial for performance as our parameter choice of $(\theta, \ell_0/\theta) = (6, 64)$ results in 4.43× (resp. 1.38×) faster latency compared to when using only the column method (resp. mux method) with $(\theta, \ell_0/\theta) = (350, 1)$ (resp, $(\theta, \ell_0/\theta) = (1, 512)$). Also, based on the lefthand figure, by choosing an adequate $B = 4$ for the base-B decomposition of mux rotation, we could outperform $B = 2$ or $B = 8$ when $(\theta, \ell_0/\theta) = (6, 64)$.

Bootstrapping bits. We constructed an even faster parameter set for boostrapping bits, as we can use much smaller base primes. We designed Bit13, with which bootstrapping can be performed in mere 174ms with 32 cores (see Table 5). When comparing with [BCKS24], the state-of-the-art method for bootstrapping bits, SHIP (Bit13) shows slightly higher latency in single-core executions. How-

Table 5. Single-core performance of bootstrapping bits. We compare SHIP with [BCKS24], which we measured its performance on our system, and the reported latency of [LMSS23] and [CGGI16b]. For SHIP and [BCKS24], we used a sparse secret with $h = 256$ as the main secret and switched to $h = 31$ at the lowest level.

Scheme	Param. set	N	$\log PQ$	dnum	(Avg. prime bits) × (# of primes)					BlindRot		
					Base	S2C	Mult	Boot	Aux	θ	ℓ_0/θ	B
SHIP	Bit13	2^{13}	205	4	18 × 1	17 × 1	17 × 1	21 × 5	24 × 2	6	64	4
[BCKS24]	Param14	2^{14}	424	13	32 × 1	28 × 1	26 × 2	31 × 9	32 × 1	-		

Scheme	N	# of slots	# of mult levels	Latency	
				1-core	32-core (multi-core speedup)
SHIP (Bit13)	2^{13}	2^{12}	1	2.06s	**174ms (11.8×)**
[BCKS24] (Param14)	2^{14}	2^{13}	2(1)†	1.43s	379ms (3.77×)
[LMSS23]	2^{10}	1	-	6.49ms	-
[CGGI16b]	2^{10}	1	-	10.5ms	-

† One level is removed to decrease the high bootstrapping failure probability.

ever, thanks to the high multi-core speedup reaching 11.8×, we obtain 2.18× << faster bootstrapping time when using 32 cores.

Also, although DM/CGGI-based [LMSS23] and [CGGI16b] show much better latency compared to CKKS-based schemes, they lack the slot packing capability and exhibit relatively low throughput. In terms of single-core throughput, SHIP (Bit13) becomes a favorable solution compared to [LMSS23] when more than only 318 slots out of its 4096 slots are effectively utilized.

Acknowledgements. We thank the anonymous reviewers for their comments, Seonhong Min for his careful proofreading, and Hyeongmin Choe for useful discussions. All of them helped us improve this paper.

References

[ACJ24] R. Agrawal, A. Chandrakasan, and A. Joshi. HEAP: A fully homomorphic encryption accelerator with parallelized bootstrapping. In *ISCA*, 2024.

[APS15] M. R. Albrecht, R. Player, and S. Scott. On the concrete hardness of learning with errors. *J. Math. Cryptol.*, 2015. Software available at `https://github.com/malb/lattice-estimator` (commit 14a3625).

[BCK+23] Y. Bae, J. H. Cheon, J. Kim, J. H. Park, and D. Stehlé. HERMES: Efficient ring packing using MLWE ciphertexts and application to transciphering. In *CRYPTO*, 2023.

[BCKS24] Y. Bae, J. H. Cheon, J. Kim, and D. Stehlé. Bootstrapping bits with CKKS. In *EUROCRYPT*, 2024.

[BEHZ16] J.-C. Bajard, J. Eynard, M. A. Hasan, and V. Zucca. A full RNS variant of FV like somewhat homomorphic encryption schemes. In *SAC*, 2016.

[BGGJ20] C. Boura, N. Gama, M. Georgieva, and D. Jetchev. CHIMERA: combining ring-LWE-based fully homomorphic encryption schemes. *J. Math. Cryptol.*, 2020.

[BGV12] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In *ITCS*, 2012.

[BKSS24] Y. Bae, J. Kim, D. Stehlé, and E. Suvanto. Bootstrapping small integers with CKKS. In *ASIACRYPT*, 2024.

[BMTH21] J.-P. Bossuat, C. Mouchet, J. R. Troncoso-Pastoriza, and J.-P. Hubaux. Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys. In *EUROCRYPT*, 2021.

[Bra12] Z. Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In *CRYPTO*, 2012.

[BTH22] J.-P. Bossuat, J. R. Troncoso-Pastoriza, and J.-P. Hubaux. Bootstrapping for approximate homomorphic encryption with negligible failure-probability by using sparse-secret encapsulation. In *ACNS*, 2022.

[CCKK24] J. H. Cheon, H. Choe, M. Kang, and J. Kim. Grafting: Complementing RNS in CKKS. IACR eprint 2024/1014, 2024.

[CCS19] H. Chen, I. Chillotti, and Y. Song. Improved bootstrapping for approximate homomorphic encryption. In *EUROCRYPT*, 2019.

[CDKS21] H. Chen, W. Dai, M. Kim, and Y. Song. Efficient homomorphic conversion between (ring) LWE ciphertexts. In *ACNS*, 2021.

[CGGI16a] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *ASIACRYPT*, 2016.

[CGGI16b] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast fully homomorphic encryption library, 2016. Software available at `https://tfhe.github.io/tfhe/`.

[CGGI17] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In *ASIACRYPT*, 2017.

[CHK+18a] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song. Bootstrapping for approximate homomorphic encryption. In *EUROCRYPT*, 2018.

[CHK+18b] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song. A full RNS variant of approximate homomorphic encryption. In *SAC*, 2018.

[CKKL24] H. Chung, H. Kim, Y.-S. Kim, and Y. Lee. Amortized large look-up table evaluation with multivariate polynomials for homomorphic encryption. IACR eprint 2024/274, 2024.

[CKKS17] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, 2017.

[Cry22] CryptoLab. HEaaN library, 2022. Available at `https://heaan.it/`.

[DM15] L. Ducas and D. Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, 2015.

[DMPS24] N. Drucker, G. Moshkowich, T. Pelleg, and H. Shaul. BLEACH: Cleaning errors in discrete computations over CKKS. *J. Cryptol.*, 2024.

[FV12] J. Fan and F. Vercauteren. Somewhat practical fully homomorphic encryption. IACR eprint 2012/144, 2012.

[GPL23] A. Guimarães, H. V. L. Pereira, and B. Van Leeuwen. Amortized bootstrapping revisited: Simpler, asymptotically-faster, implemented. In *ASIACRYPT*, 2023.

[HHC19] K. Han, M. Hhan, and J. H. Cheon. Improved homomorphic discrete Fourier transforms and FHE bootstrapping. *IEEE Access*, 2019.

[HK20] K. Han and D. Ki. Better bootstrapping for approximate homomorphic encryption. In *CT-RSA*, 2020.

[HS14] S. Halevi and V. Shoup. Algorithms in HElib. In *CRYPTO*, 2014.

[HS15] S. Halevi and V. Shoup. Bootstrapping in HELib. In *EUROCRYPT*, 2015.

[HS18] S. Halevi and V. Shoup. Faster homomorphic linear transformations in HElib. In *CRYPTO*, 2018.

[KDE+24] A. Kim, M. Deryabin, J. Eom, R. Choi, Y. Lee, W. Ghang, and D. Yoo. General bootstrapping approach for RLWE-based homomorphic encryption. *IEEE Trans. Computers*, 2024.

[KPK+22] S. Kim, M. Park, J. Kim, T. Kim, and C. Min. EvalRound algorithm in CKKS bootstrapping. In *ASIACRYPT*, 2022.

[KSS24] J. Kim, J. Seo, and Y. Song. Simpler and faster BFV bootstrapping for arbitrary plaintext modulus from CKKS. In *CCS*, 2024.

[LLK+22] Y. Lee, J.-W. Lee, Y.-S. Kim, Y. Kim, J.-S. No, and H. Kang. High-precision bootstrapping for approximate homomorphic encryption by error variance minimization. In *EUROCRYPT*, 2022.

[LLL+21] J.-W. Lee, E. Lee, Y. Lee, Y.-S. Kim, and J.-S. No. High-precision bootstrapping of RNS-CKKS homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In *EUROCRYPT*, 2021.

[LMSS23] C. Lee, S. Min, J. Seo, and Y. Song. Faster TFHE bootstrapping with block binary keys. In *AsiaCCS*, 2023.

[LPR10] V. Lyubashevsky, C. Peikert, and O. Regev. On ideal lattices and learning with errors over rings. In *EUROCRYPT*, 2010.

[LW23a] F.-H. Liu and H. Wang. Batch bootstrapping I: A new framework for SIMD bootstrapping in polynomial modulus. In *EUROCRYPT*, 2023.

[LW23b] F.-H. Liu and H. Wang. Batch bootstrapping II: bootstrapping in polynomial modulus only requires $\widetilde{O}(1)$ FHE multiplications in amortization. In *EUROCRYPT*, 2023.

[May21] A. May. How to meet ternary LWE keys. In *CRYPTO*, 2021.

[MHWW24a] S. Ma, T. Huang, A. Wang, and X. Wang. Accelerating BGV bootstrapping for large p using null polynomials over $\mathbb{Z}_{p^e}$. In *EUROCRYPT*, 2024.

[MHWW24b] S. Ma, T. Huang, A. Wang, and X. Wang. Faster BGV bootstrapping for power-of-two cyclotomics through homomorphic NTT. In *ASIACRYPT*, 2024.

[MKMS24] G. De Micheli, D. Kim, D. Micciancio, and A. Suhl. Faster amortized FHEW bootstrapping using ring automorphisms. In *PKC*, 2024.

[MS18] D. Micciancio and J. Sorrell. Ring packing and amortized FHEW bootstrapping. In *ICALP*, 2018.

[OPP23] H. Okada, R. Player, and S. Pohmann. Homomorphic polynomial evaluation using Galois structure and applications to BFV bootstrapping. In *ASIACRYPT*, 2023.

[Sko23] M. Skorski. Bernstein-type bounds for beta distribution. *Modern Stochastics: Theory and Applications*, 2023.

[SSKM24] H. Sung, S. Seo, T. Kim, and C. Min. Evalround+ bootstrapping and its rigorous analysis for CKKS scheme. IACR eprint 2024/1379, 2024.

[SSTX09] D. Stehlé, R. Steinfeld, K. Tanaka, and K. Xagawa. Efficient public key encryption based on ideal lattices. In *ASIACRYPT*, 2009.

[Zam24] Zama. TFHE-rs: A pure rust implementation of the TFHE scheme for boolean and integer arithmetics over encrypted data, 2024. Software available at `https://github.com/zama-ai/tfhe-rs`.

Supplementary Material

A Proof of Lemma 1

We restate Lemma 1 below, in an extended form that also considers the second halves $\boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j)$ of the $\omega^{(\mathbf{a}_j)_i}$'s. We do not need them in our bootstrapping algorithm, but provide the result for the sake of completeness.

Lemma 4. *Let* $\mathbf{a} \in \mathbb{Z}^N$, $0 \leqslant j < N$ *and* $s_j \in \{-1, 1\}$. *Let* $\tilde{\mathbf{a}}_j \in \mathbb{Z}^N$ *be defined by* $\sum_{0 \leqslant i < N} (\tilde{\mathbf{a}}_j)_i X^i = s_j X^j \sum_{0 \leqslant i < N} a_i X^i \bmod X^N + 1$. *Then we have:*

- *for* $0 \leqslant j < N/2$,

$$\begin{aligned}
\boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j) &= \mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}_j + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}_j \\
&\quad + \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \frac{1+s_j}{2}(\mathbf{1}-\mathbf{m}_j) + \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \frac{1-s_j}{2}(\mathbf{1}-\mathbf{m}_j) \ , \\
\boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j) &= \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}_j + \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}_j \\
&\quad + \mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \frac{1+s_j}{2}(\mathbf{1}-\mathbf{m}_j) + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \frac{1-s_j}{2}(\mathbf{1}-\mathbf{m}_j) \ ,
\end{aligned}$$

- *for* $N/2 \leqslant j < N$, *using the notation* $j' = j - N/2$,

$$\begin{aligned}
\boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j) &= \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}_{j'} + \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}_{j'} \\
&\quad + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \frac{1+s_j}{2}(\mathbf{1}-\mathbf{m}_{j'}) + \mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \frac{1-s_j}{2}(\mathbf{1}-\mathbf{m}_{j'}) \ , \\
\boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j) &= \mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}_{j'} + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}_{j'} \\
&\quad + \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \frac{1+s_j}{2}(\mathbf{1}-\mathbf{m}_{j'}) + \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \frac{1-s_j}{2}(\mathbf{1}-\mathbf{m}_{j'}) \ .
\end{aligned}$$

Proof. We first prove the formulas for $\omega_1(\tilde{\mathbf{a}}_j)$. We have, modulo $X^N + 1$:

$$s_j X^j \sum_{0 \leqslant i < N} a_i X^i = \sum_{0 \leqslant i < N} s_j a_i X^{i+j} = \sum_{0 \leqslant i < N} s_j a_{i-j} \alpha_{ij} X^i \ ,$$

where the indices of the coefficients of $\mathbf{a}$ are to be taken modulo N, and $\alpha_{ij} = 1$ if $i \geqslant j$ and $\alpha_{ij} = -1$ otherwise. In view of this, we have $(\tilde{\mathbf{a}}_j)_i = s_j a_{i-j} \alpha_{ij}$ for all $0 \leqslant i < N$. Hence, using the fact that $s_j \in \{-1, 1\}$, we have (without modular reduction in the indices of the coefficients of $\mathbf{a}$):

$$\omega^{(\tilde{\mathbf{a}}_j)_i} = \omega^{s_j a_{i-j}} = \frac{1+s_j}{2}\omega^{a_{i-j}} + \frac{1-s_j}{2}\omega^{-a_{i-j}} \qquad \text{if } i \geqslant j \ , \tag{10}$$

$$\omega^{(\tilde{\mathbf{a}}_j)_i} = \omega^{-s_j a_{N+i-j}} = \frac{1+s_j}{2}\omega^{-a_{N+i-j}} + \frac{1-s_j}{2}\omega^{a_{N+i-j}} \qquad \text{if } i < j \ . \tag{11}$$

Assume that $j \leqslant i < N/2$. The value $\omega^{a_{i-j}}$ is the i-th coordinate of the vector $\mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a}))$ and the value $\omega^{-a_{i-j}}$ is the i-th coordinate of the vector $\mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a}))$. As such, Equation (10) gives:

$$\mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}_j + \mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}_j = (\mathbf{0}_j, (\omega^{(\tilde{\mathbf{a}}_j)_i})_{j \leqslant i < N/2}) \ .$$

Assume that $i < j < N/2$. The value $\omega^{-a_{N+i-j}} = \omega^{-a_{N/2+i-(j-N/2)}}$ is the i-th term of $\mathsf{Rot}_{j-N/2}(\boldsymbol{\omega}_2(-\mathbf{a})) = \mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a}))$ (this equality holds as the index of Rot is defined modulo $N/2$). Equation (11) thus gives:

$$\mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) \odot \frac{1+s_j}{2}\mathbf{m}'_j + \mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a})) \odot \frac{1-s_j}{2}\mathbf{m}'_j = ((\omega^{(\tilde{\mathbf{a}}_j)_i})_{0 \leqslant i < j}, \mathbf{0}_{N/2-j}) \ ,$$

with $\mathbf{m}'_j = \mathbf{1} - \mathbf{m}_j$. By adding the last two equations, we obtain the first equation of the lemma statement.

Assume that $i < N/2 \leqslant j$. We use Equation (11). For $N + i - j \geqslant N/2$ (or, equivalently, $i \geqslant j - N/2$), the value $\omega^{(\tilde{\mathbf{a}}_j)_i}$ is the i-th term of $\frac{1+s_j}{2}\mathsf{Rot}_j(\boldsymbol{\omega}_2(-\mathbf{a})) + \frac{1-s_j}{2}\mathsf{Rot}_j(\boldsymbol{\omega}_2(\mathbf{a}))$. For $N + i - j < N/2$ (or, equivalently, $i < j - N/2$), it is the i-th term of $\frac{1+s_j}{2}\mathsf{Rot}_j(\boldsymbol{\omega}_1(-\mathbf{a})) + \frac{1-s_j}{2}\mathsf{Rot}_j(\boldsymbol{\omega}_1(\mathbf{a}))$, which gives the third equation of the lemma statement.

We now turn to the statements focusing on $\omega_2(\tilde{\mathbf{a}}_j)$. For $j < N/2$, we define $\hat{\mathbf{a}}_j$ as $\hat{\mathbf{a}}_j = X^{N/2} \cdot \tilde{\mathbf{a}}_j \bmod (X^N + 1)$. In particular, we have $(\hat{\mathbf{a}}_j)_i = -(\tilde{\mathbf{a}}_j)_{i+N/2}$ for $0 \leqslant i < N/2$ and $(\hat{\mathbf{a}}_j)_i = (\tilde{\mathbf{a}}_j)_{i-N/2}$ for $N/2 \leqslant i < N$. This implies that $\boldsymbol{\omega}_1(-\hat{\mathbf{a}}_j) = \boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j)$ and $\boldsymbol{\omega}_2(\hat{\mathbf{a}}_j) = \boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j)$. We deduce from the equation above and the definition of $\tilde{\mathbf{a}}_j$ that the lemma statement on $\boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j)$ can be derived from the one on $\boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j)$ with rotation index $j + N/2$ and $\mathbf{a}$ replaced by $-\mathbf{a}$.

For $j > N/2$, we define $\breve{\mathbf{a}}_j$ as $\breve{\mathbf{a}}_j = X^{-N/2} \cdot \tilde{\mathbf{a}}_j \bmod (X^N + 1)$ and observe that $\boldsymbol{\omega}_1(\breve{\mathbf{a}}_j) = \boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j)$ and $\boldsymbol{\omega}_2(-\breve{\mathbf{a}}_j) = \boldsymbol{\omega}_1(\tilde{\mathbf{a}}_j)$. The lemma statement on $\boldsymbol{\omega}_2(\breve{\mathbf{a}}_j)$ can hence be derived from the one on $\boldsymbol{\omega}_2(\tilde{\mathbf{a}}_j)$ with rotation index $j - N/2$. □

B Proof of Lemma 2

By definition of a' and b', there exist $I' \in R$ and a small ε' in R such that

$$a' \cdot \mathsf{sk}(X^{5^j}) + b' = \mathsf{Ecd}(\mathsf{Rot}_j(\boldsymbol{\mu})) + QI' + \varepsilon' \ .$$

This equation holds over R and hence also over R_{PQ}.

Now, over R_{PQ}, we have:

$$\begin{aligned} a'(a_0 \cdot \mathsf{sk} + b_0) + b'(a_1 \cdot \mathsf{sk} + b_1) &= a'\left(P\beta \cdot \mathsf{sk}(X^{5^j}) + \varepsilon_0\right) + b'(P\beta + \varepsilon_1) \\ &= P\beta\left(a' \cdot \mathsf{sk}(X^{5^j}) + b'\right) + a'\varepsilon_0 + b'\varepsilon_1 \\ &= P\beta\left(\mathsf{Ecd}(\mathsf{Rot}_j(\boldsymbol{\mu})) + QI' + \varepsilon'\right) + a'\varepsilon_0 + b'\varepsilon_1 \\ &= P\beta \cdot \mathsf{Ecd}(\mathsf{Rot}_j(\boldsymbol{\mu})) + a'\varepsilon_0 + b'\varepsilon_1 + P\beta\varepsilon' \ . \end{aligned}$$

Dividing by P and rounding, we obtain that:

$$\mathsf{Dec}_{\mathsf{sk}}\left(\mathsf{HMuxRot}^{(j)}\left(\mathsf{hmrkey}_{\beta}^{(j)}, \mathsf{ct}\right)\right) = \beta \cdot \mathsf{Ecd}\left(\mathsf{Rot}_j(\boldsymbol{\mu})\right) + e \ .$$

In the above, the error term e is defined as

$$e = P^{-1}((a'\varepsilon_0 + b'\varepsilon_1)) + \beta\varepsilon' + e_{\mathrm{rnd},0} \cdot \mathsf{sk} + e_{\mathrm{rnd},1} \ ,$$

where $e_{\mathrm{rnd},0}$ (resp. $e_{\mathrm{rnd},1}$) is error due to rounding the "a-part" (resp. "b-part") to R. Since $P \geqslant Q$ and $\mathsf{sk} \in R$ is small, this error term is expected to be small compared with the message. □